RTOS G2D

开发指南

版本号: 2.0

发布日期: 2026.2.11

深圳易新泰科技有限公司YiXinTai

版本历史

版本号	日期	制/修订人	内容描述
1.0	2020.7.29	AWA1639	初始版本
1.0	2021.4.10	AWA1693	添加输出宽度限制说明
2.0	2026.2.11	AWA2257	规范化文档内容，新增 MB32 格式转换部分内容

深圳易新泰科技有限公司YiXinTai

版权所有 © 珠海全志科技股份有限公司。保留一切权利

文档问题反馈: aw-document@allwinnertech.com

目录

第 1 章 前言 1
1.1 文档简介 1
1.2 目标读者 1
1.3 适用范围 1
1.4 G2D 驱动版本（按硬件 IP 适配） 1
1.5 文档约定 1
1.5.1 标志说明 1
1.5.2 数值单位约定 2
1.6 相关术语 2

第 2 章 概述 3
2.1 G2D 模块功能概述 3
2.1.1 旋转和镜像 (Rotate/Mirror) 3
2.1.2 图像裁剪和拼接 (Crop/Stiching) 3
2.1.3 矩形填充 (fill color rectgngle) 4
2.1.4 图层混合 (alpha blending) 4
2.1.5 色键 (colorkey) 5
2.1.6 缩放 (Stretchblt) 5
2.1.7 二元光栅操作 (Rop2) 6
2.1.8 三元光栅操作 (Rop3) 7
2.2 图像格式支持 8
2.3 限制条件 8
2.3.1 地址对齐要求 8
2.3.2 宽高对齐要求 9
2.3.3 最小最大宽高要求 9
2.3.3.1 最小宽高 9
2.3.3.2 最大宽高 9
2.3.4 输出格式显示 9

第 3 章 G2D 模块驱动开发 10
3.1 G2D 驱动框架 10
3.2 源码目录 10
3.2.1 g2d_rcq 目录（适配 g2d1XX IP） 10
3.2.2 g2d200 目录（适配 g2d2XX IP） 11
3.3 配置方式 11
3.4 测试用例 12

第 4 章 数据结构 13
4.1 g2d_blt_flags 13
4.2 g2d_fillrect_flags 13
4.3 g2d_data_fmt(version 1.0) 14
4.4 g2d_pixel_seq(version 1.0) 15
4.5 g2d_blt_flags_h 16
4.6 g2d_image(version 1.0) 17
4.7 g2d_image_enh 18
4.8 g2d_fmt_enh 18
4.9 g2d_rop3_cmd_flag 19
4.10 g2d_bld_cmd_flag 20
4.11 g2d_ck 20
4.12 g2d_alpha_mode_enh 21
4.13 g2d_color_gmt 21
4.14 g2d_scan_order(version 1.0) 21
4.15 g2d_blt(version 1.0) 22
4.16 g2d_fillrect(version 1.0) 22

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

4.17 g2d_stretchblt(version 1.0) . . . 22
4.18 g2d_blt_h . . . 23
4.19 g2d_bld(version 1.0) . . . 23

第 5 章 G2D 模块接口和使用 Demo 24
5.1 旋转和镜像 (Rotate/Mirror) . . . 24
5.1.1 关键参数配置 . . . 24
5.1.2 使用 demo . . . 25
5.2 图像裁剪和拼接 (Crop/Stiching) . . . 26
5.2.1 关键参数配置 . . . 26
5.2.2 使用 demo . . . 27
5.3 图像缩放或下采样 (Scaler/Down_Sampling) . . . 31
5.3.1 关键参数配置 . . . 31
5.3.2 使用 demo . . . 31
5.4 图像格式转换 (Format_Conversion) . . . 32
5.4.1 关键参数配置 . . . 32
5.4.2 使用 demo . . . 34
5.5 图层混合 (Alpha Blending) . . . 34
5.5.1 关键参数配置 . . . 34
5.5.2 使用 demo . . . 35
5.6 画点/画线/矩形填充 (Point/Line Drawing/Rect_ColorFilling) . . . 36
5.6.1 关键参数配置 . . . 36
5.6.2 使用 demo . . . 36
5.7 色键 (ColorKey) . . . 37
5.7.1 关键结构 . . . 37
5.7.2 使用 demo . . . 37
5.8 二元光栅操作 (Rop2) . . . 38
5.8.1 关键参数配置 . . . 38
5.8.2 使用 demo . . . 39
5.9 三元光栅操作 (Rop3) . . . 39
5.9.1 关键参数配置 . . . 39
5.9.2 使用 demo . . . 40
5.10 批处理工作模式 . . . 41
5.10.1 关键参数配置 . . . 41
5.10.2 批处理工作同步模式使用 demo . . . 42
5.10.2.1 G2D_CMD_MIXER_TASK . . . 42
5.10.3 批处理工作异步模式使用 demo . . . 44
5.10.3.1 G2D_CMD_CREATE_TASK . . . 44
5.10.3.2 G2D_CMD_TASK_APPLY . . . 44
5.10.3.3 G2D_CMD_TASK_DESTROY . . . 45
5.10.3.4 G2D_CMD_TASK_GET_PARA . . . 45
5.11 操作混合 . . . 45

第 6 章 FAQ 48
6.1 G2D 理论性能计算公式 . . . 48
6.2 G2D 调试节点 . . . 48

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

插图

图 2-1 rotate . . . 3
图 2-2 clip size 示意图 . . . 4
图 2-3 fill rectangle 示意图 . . . 4
图 2-4 alpha blending 示意图 . . . 5
图 2-5 colorkey 示意图 . . . 5
图 2-6 scale and alpha blending 示意图 . . . 6
图 2-7 maskblt rop3 . . . 7
图 3-1 G2D 驱动框架 . . . 10
图 5-1 mixerpara . . . 41

深圳易新泰科技有限公司YiXinTai

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

第 1 章 前言

1.1 文档简介

介绍 Sunxi 平台 RTOS 上 G2D 驱动 hal 的一般使用方法及调试接口，为开发与调试提供参考。

1.2 目标读者

G2D 驱动及应用层的开发/维护人员。

1.3 适用范围

表 1-1: 适用产品列表

产品名称	硬件版本	内核版本	驱动文件
V833	G2D101	Melis	rtos-hal/hal/source/g2d_rcq/
F133	G2D110	Melis	rtos-hal/hal/source/g2d_rcq/
R128	G2D101	FreeRTOS	rtos-hal/hal/source/g2d_rcq/
XR875	G2D101	FreeRTOS	rtos-hal/hal/source/g2d_rcq/
F102	G2D200	Melis	rtos-hal/hal/source/g2d_200/

1.4 G2D 驱动版本（按硬件 IP 适配）

RTOS G2D 驱动源码目前一共有两套，分别是 g2d_rcq 和 g2d200。详情见下面表格。g2d1XX(如 g2d101 版本 ip) 使用 g2d_rcq。g2d200 版本 ip 使用 g2d200 驱动。

接口兼容性与编译配置如下：

G2D 驱动版本	适配场景	编译配置
g2d_rcq	适配 g2d1XX 版本 IP	CONFIG_DRIVERS_G2D
g2d200	适配 g2d2XX 版本 IP	CONFIG_DRIVER_G2D_200，CONFIG_G2D_ROTATE,CONFIG_G2D_MIXER

1.5 文档约定

1.5.1 标志说明

注意

提醒操作中应注意的事项。不当的操作可能会损坏器件，影响可靠性、降低性能等。

说明

为准确理解文中指令、正确实施操作而提供的补充或强调信息。

技巧

一些容易忽视的小功能、技巧。了解这些功能或技巧能帮助解决特定问题或者节省操作时间。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

1.5.2 数值单位约定

本文档在描述数据容量（如 NAND 容量）时，单位词头代表的是 1024 的倍数；描述频率、数据速率等时则代表的是 1000 的倍数。具体如下：

数据容量（如 NAND 容量）

```
1 K        1024
1 M        1 048 576
1 G        1 073 741 824
```

频率，数据速率等

```
1 k        1000
1 M        1 000 000
1 G        1 000 000 000
```

1.6 相关术语

术语	释义
Rotate/Mirror	图像旋转或镜像处理
Crop/Stitching	图像裁剪与拼接操作
Scaler/Down_Sampling	图像缩放/图像数据下采样，比如 Y8 每隔一行、一列采样一次，2048*2048->1024/1024
PixelFormat_Conversion	像素格式转换，比如 ARGB8888 转 NV12
ColorSpace_Conversion	颜色空间转换，比如 RGB BT709 转 YUV BT601
Point/Line Drawing Rect_ColorFilling	画点/画线，向矩形区域填充指定颜色
ColorKey	两个图像叠加混合的时候，对特殊色做特殊过滤，符合条件的区域叫 match 区，在 match 区就全部使用另外一个图层的颜色值，不符合条件的区域就是非 match 区，非 match 区就是走普通的 alpha 混合
Alpha Blending	对任意两张图像可以合成为一张图像，合成图像的像素取值根据数学公式：RGB3 = (1- a) * RGB1 + a * RGB2
ROI	感兴趣区域（Region of Interest），通常用 ROI(x, y, w, h) 来表示。在图像处理领域，ROI 指的是对图像中的特定区域感兴趣，而不是整个图像。通过指定 ROI，可以针对图像中的特定区域进行操作，例如裁剪、处理或提取感兴趣的信息。x 表示矩形区域的左上角点的 x 坐标，y 表示矩形区域的左上角点的 y 坐标，w 表示矩形区域的宽度（width）h 表示矩形区域的高度（height）
RCQ/AHB	RCQ，AHB 为两种更新寄存器的方式，RCQ 为硬件自更新寄存器，能够下发批处理任务，节省中断响应时间

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

第 2 章 概述

G2D（Graphics 2D Engine，2D 图形引擎）模块是 Allwinner sunxi 平台集成的硬件加速单元，主要提供以下图形处理加速功能：

- 图像变换：旋转、镜像、裁剪、拼接
- 缩放与采样：无极缩放、图像数据下采样（如 Y8 格式隔行隔列采样，实现 2048×2048 至 1024×1024 的缩放）
- 格式转换：像素格式转换（如 ARGB8888 转 NV12）、颜色空间转换（如 RGB BT709 转 YUV BT601）
- 图层与绘制：图层合成、ColorKey 处理、矩形区域颜色填充、点 / 线绘制
- 光栅操作：二元光栅操作、三元光栅操作

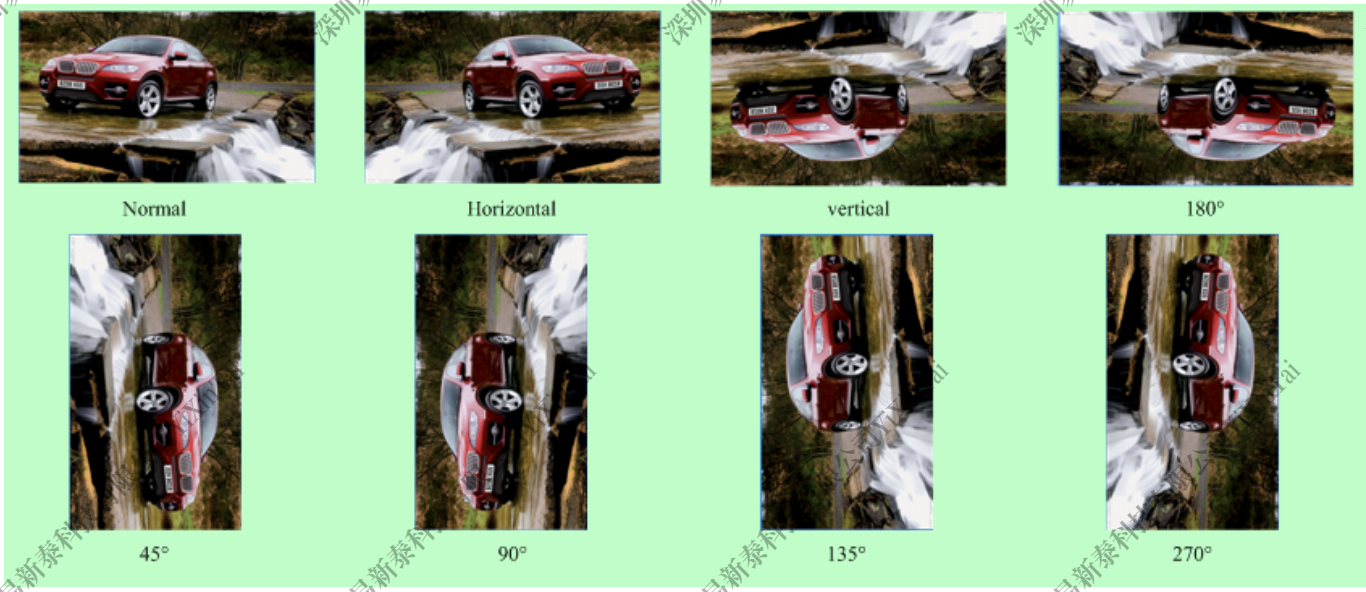
2.1 G2D 模块功能概述

本节介绍 G2D 模块核心功能的原理与处理效果。

2.1.1 旋转和镜像 (Rotate/Mirror)

旋转镜像主要实现如下 8 种操作：Horizontal-Flip、Vertical-Flip、Rotate0°、Rotate90°、Rotate180°、Rotate270°、Rotate90°+Horizontal-Flip、Rotate90°+Vertical-Flip。

图 2-1: rotate



2.1.2 图像裁剪和拼接 (Crop/Stiching)

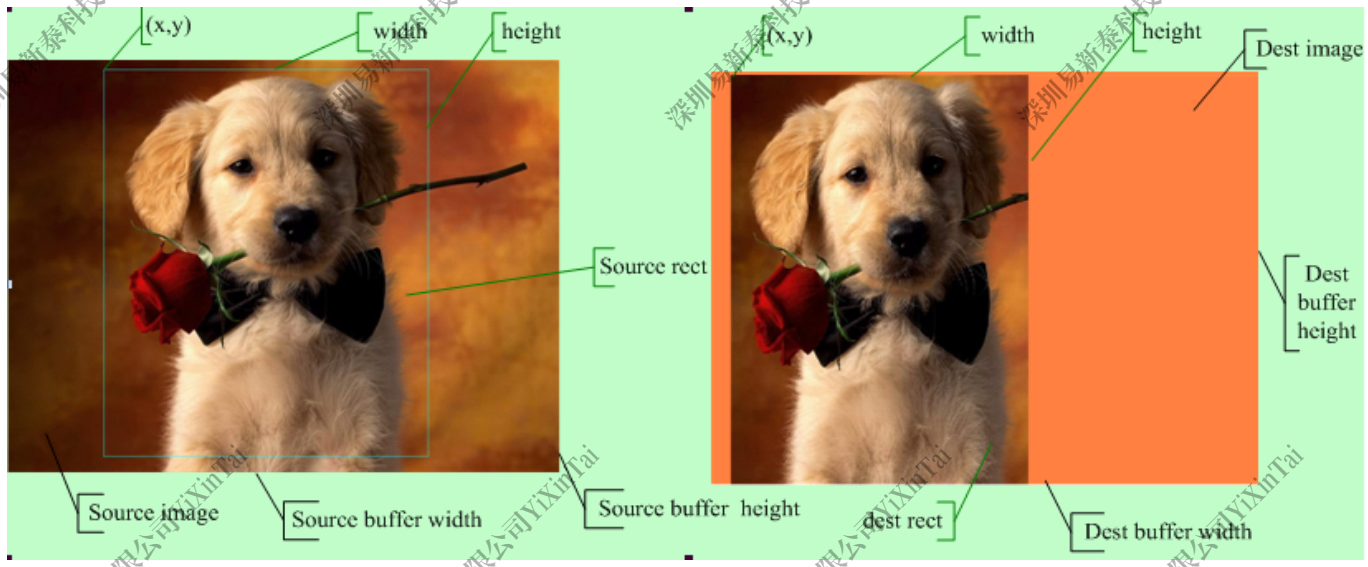
图层的 size 相关的参数有 Image size、source rect 以及 dest rect。

- Image size 指图片的 buffer 属性，可以理解为图片的原始完整的大小；
- source rect 是指图片 clip 区域的位置与尺寸（G2D 驱动支持 clip 完整的图片，也可以 clip 图片中某一块区域）；
- dest rect 则为图片 dest Image 在显示屏幕中的位置与尺寸。如果是 Stretchblt，source rect 与 dest rect 的宽高可以不一样，其他工作模式的图层，这二者应该一致。

G2D 可裁剪输入源图的（x，y，w，h）指定区域，如下图所示，左图区域为完整的图片尺寸，淡绿色矩形区域则为图片 clip 区域，即 source rect；右图橙黄色区域为 dest image，little dog 区域则为 source rect 拷贝到 dest image 区域的 dest rect。

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

图 2-2: clip size 示意图



技巧

g2d200 版本开始支持单列裁剪

2.1.3 矩形填充 (fill color rectgngle)

填充矩形区域功能可以实现对某块区域进行预订的颜色值填充，如下图就填充了 0xFF0080FF 的 ARGB 值，该功能还可以通过设定数据区域大小实现画点和直线，同时也可以通过设定 flag 实现一种填充颜色和目标做 alpha 运算。

图 2-3: fill rectangle 示意图



2.1.4 图层混合 (alpha blending)

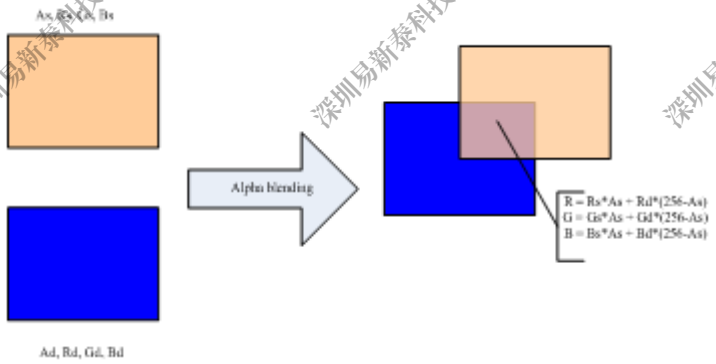
不同的图层之间可以做 alpha blending。Alpha 分为 pixel alpha、plane alpha、multi alpha 三种：

- pixel alpha 意为每个像素自带有一个专属 alpha 值。
- plane alpha 则是一个图层中所有像素共用一个 globe alpha 值。
- multi alpha 则每个像素在代入 alpha 运算时的值为 globe alpha*pixel alpha，可以通过 G2D 驱动接口的 flag 去控制。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

文档问题反馈: aw-document@allwinnertech.com

图 2-4: alpha blending 示意图



⚠ 注意

- 不支持两张 yuv 格式做 blend 图层混合操作

2.1.5 色键 (colorkey)

不同 image 之间可以做 colorkey 效果，效果如下。

- 左图中 destination 的优先级高于 source，destination 中 match 部分（橙色五角星部分），则被选择透过，显示为 source 与 destination 做 alpha blending 后的效果图。
- 右图中 source 的优先级高于 destination，则 source 中 match 部分（深红色五角星部分），则被选择透过，直接显示 destination 与 source 做 alpha blending 后的效果图。

图 2-5: colorkey 示意图



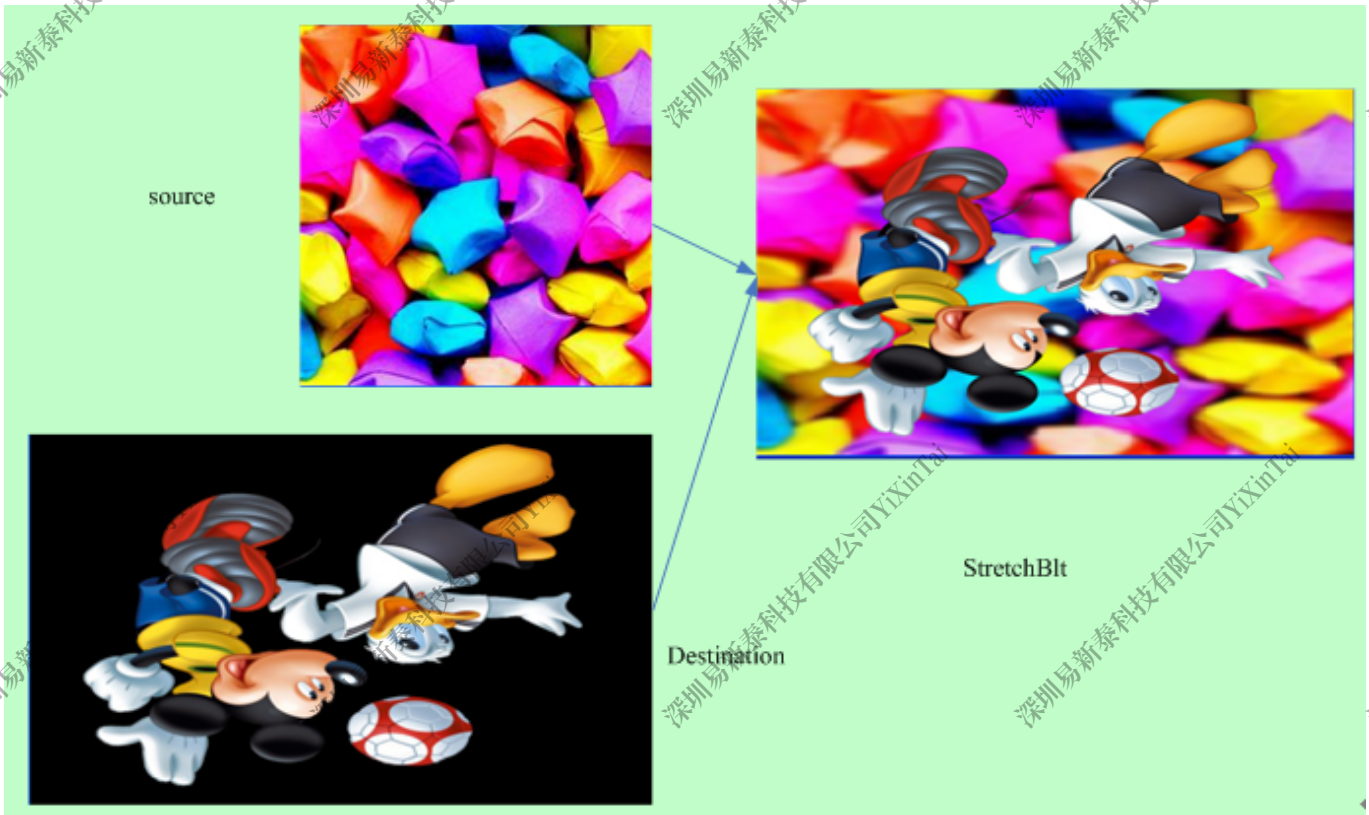
2.1.6 缩放 (Stretchblt)

缩放主要是把 source 按照 destination 的 size 进行缩放，并最终与 destination 做 Alpha blending、colorkey 等运算或直接旋转镜像后拷贝到目标。

深圳易新泰科技有限公司YiXinTai

版权所有 © 珠海全志科技股份有限公司。保留一切权利

图 2-6: scale and alpha blending 示意图



2.1.7 二元光栅操作 (Rop2)

二元光栅操作（Binary Raster Operations）是图形处理中常见的一种光栅操作方式，使用两个输入值进行操作。在二元光栅操作中，通常表示为 [src] op [dst] -> [dst]，其中包括源图像（src）、目标图像（dst）和结果保存在目标图像中。

二元光栅操作的基本原理是通过对源图像中的像素与目标图像中对应位置的像素进行某种操作来生成最终的输出。这种操作可以是简单的像素值覆盖、逻辑运算、位运算等，用于实现各种图形处理效果和算法。

G2D 目前支持的二元光栅操作和对应操作码见下面的表格

操作码	含义
G2D_BLT_BLACKNESS	dst = blackness
G2D_BLT_NOTMERGEPEN	dst = ~(dst + src)
G2D_BLT_MASKNOTPEN	dst = ~src & dst
G2D_BLT_NOTCOPYPEN	dst = ~src
G2D_BLT_MASKPENNOT	dst = src & ~dst
G2D_BLT_NOT	dst = ~dst
G2D_BLT_XORPEN	dst = src ^dst
G2D_BLT_NOTMASKPEN	dst = ~(src & dst)
G2D_BLT_MASKPEN	dst = src & dst
G2D_BLT_NOTXORPEN	dst = ~(src ^dst)
G2D_BLT_NOP	dst = dst
G2D_BLT_MERGENOTPEN	dst = ~src + dst
G2D_BLT_COPYPEN	dst = src
G2D_BLT_MERGEPENNOT	dst = src + ~dst

深圳易新泰科技有限公司YiXinTai

版权所有 © 珠海全志科技股份有限公司。保留一切权利

文档问题反馈: aw-document@allwinnertech.com

操作码	含义
G2D_BLT_MERGEPEN	dst = src + dst
G2D_BLT_WHITENESS	dst = whiteness

2.1.8 三元光栅操作 (Rop3)

三元光栅操作（Ternary Raster Operations）是图形处理中使用三个输入值进行操作的一种光栅操作方式。在三元光栅操作中，通常表示为 [src0] op [src1] op [src2] -> [dst]，其中包括两个源图像（src0 和 src1）、一个额外参数（src2），以及目标图像（dst）来保存操作结果。

三元光栅操作通常用于更复杂的图形处理任务，需要对多个输入进行组合、运算或控制的情况。这种操作方式可以实现更加灵活和多样化的图形处理效果，扩展了图形处理的功能和应用范围。

一些常见的三元光栅操作包括：

1. 混合模式：通过将两个源图像按照第三个参数指定的比例混合，实现不同程度的图像叠加效果。
2. 遮罩操作：使用第三个参数作为掩码（mask），指定哪些像素需要参与操作，实现局部区域的处理。
3. 调色板操作：利用第三个参数作为调色板索引，实现颜色映射或特定颜色的替换。

下图展示了三元光栅操作中的遮罩操作，从左到右，从上到下，依次为 src0, src1, src2, dst。

图 2-7: maskblt rop3



G2D 支持的三元光栅操作和对应操作码见下面的表格

操作码	含义
G2D_ROP3_BLACKNESS	dst = BLACK
G2D_ROP3_NOTSRCERASE	dst = (NOT src) AND (NOT dst)

版权所有 © 珠海全志科技股份有限公司。保留一切权利

操作码	含义
G2D_ROP3_NOTSRCCOPY	dst = (NOT src)
G2D_ROP3_SRCERASE	dst = src AND (NOT dst)
G2D_ROP3_DSTINVERT	dst = (NOT dst)
G2D_ROP3_PATINVERT	dst = pattern XOR dst
G2D_ROP3_SRCINVERT	dst = src XOR dst
G2D_ROP3_SRCAND	dst = src AND dst
G2D_ROP3_MERGEPAINT	dst = (NOT src) OR dst
G2D_ROP3_MERGECOPY	dst = (src AND pattern)
G2D_ROP3_SRCCOPY	dst = src
G2D_ROP3_SRCPAINT	dst = src OR dst
G2D_ROP3_PATCOPY	dst = pattern
G2D_ROP3_PATPAINT	dst = (~src OR pattern) OR DST
G2D_ROP3_WHITENESS	dst = WHITE

2.2 图像格式支持

G2D_FORMAT_ARGB8888/G2D_FORMAT_ARGB8888/G2D_FORMAT_ABGR8888/
G2D_FORMAT_RGBA8888/G2D_FORMAT_BGRA8888/G2D_FORMAT_XRGB8888,
G2D_FORMAT_XBGR8888/G2D_FORMAT_RGBX8888/G2D_FORMAT_BGRX8888/
G2D_FORMAT_RGB888/G2D_FORMAT_BGR888/G2D_FORMAT_RGB565,
G2D_FORMAT_BGR565/G2D_FORMAT_ARGB4444/G2D_FORMAT_ABGR4444/
G2D_FORMAT_RGBA4444/G2D_FORMAT_BGRA4444/G2D_FORMAT_ARGB1555,
G2D_FORMAT_ABGR1555/G2D_FORMAT_RGBA5551/G2D_FORMAT_BGRA5551/
G2D_FORMAT_ARGB2101010/G2D_FORMAT_ABGR2101010,
G2D_FORMAT_RGBA1010102/G2D_FORMAT_BGRA1010102

G2D_FORMAT_IYUV422_V0Y1U0Y0,
G2D_FORMAT_IYUV422_Y1V0Y0U0,
G2D_FORMAT_IYUV422_U0Y1V0Y0,
G2D_FORMAT_IYUV422_Y1U0Y0V0,
G2D_FORMAT_YUV422UVC_V1U1V0U0,
G2D_FORMAT_YUV422UVC_U1V1U0V0,
G2D_FORMAT_YUV422_PLANAR,
G2D_FORMAT_YUV420UVC_V1U1V0U0,
G2D_FORMAT_YUV420UVC_U1V1U0V0,
G2D_FORMAT_YUV420_PLANAR,
G2D_FORMAT_YUV411UVC_V1U1V0U0,
G2D_FORMAT_YUV411UVC_U1V1U0V0,
G2D_FORMAT_YUV411_PLANAR,
G2D_FORMAT_Y8,

G2D_FORMAT_YVU10_P010,
G2D_FORMAT_YVU10_P210,
G2D_FORMAT_YVU10_444,
G2D_FORMAT_YUV10_444,

2.3 限制条件

2.3.1 地址对齐要求

Rotate/Mirror 输入输出地址需满足 4 bytes 对齐，其他功能无限制。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

文档问题反馈: aw-document@allwinnertech.com

2.3.2 宽高对齐要求

对于以下功能：Crop/Stitching, Scaler/Down_Sampling, PixelFormat_Conversion, ColorSpace_Conversion, Point/Line Drawing, Rect_ColorFilling，RGB 格式输入，输出宽度无对齐限制，YUV 格式：YUV420 宽高都需要 2 bytes 对齐，YUV422 宽度需 2 字节对齐，YUV411 宽度需 4 字节对齐。

对于 Rotate/Mirror 功能，输入输出的宽度要 8 bytes 对齐。注意：a40i 需要满足 64 bytes 对齐，如果是 YUV 采样格式，uv plane 的宽度也需要满足对齐规则，比如 YUV420 需要宽高均 16 bytes 对齐，YUV422 宽度 16 bytes 对齐。

2.3.3 最小最大宽高要求

2.3.3.1 最小宽高

宽度上，ARGB8888 格式 ≥2，RGB888 格式 ≥3，RGB565 格式 ≥4。（单位：byte）

高度上，YUV420 格式 ≥2。

2.3.3.2 最大宽高

rotate/mirror 功能最大宽高参考平台支持列表与设计指标章节。

mixer_paramixer 功能如不涉及到裁剪和抽样，最大宽高为 2048*2048（单位：pixel），如涉及到裁剪或抽样，最大可以达到 8k 输入。注意抽样仅支持对输入 buffer 取数时操作。

2.3.4 输出格式显示

YUV 格式图像执行旋转操作时，输出格式将强制转为 YUV420，比如：I422 会被强制转换成 I420，NV16 会被强制转换成 NV12。

深圳易新泰科技有限公司YiXinTai

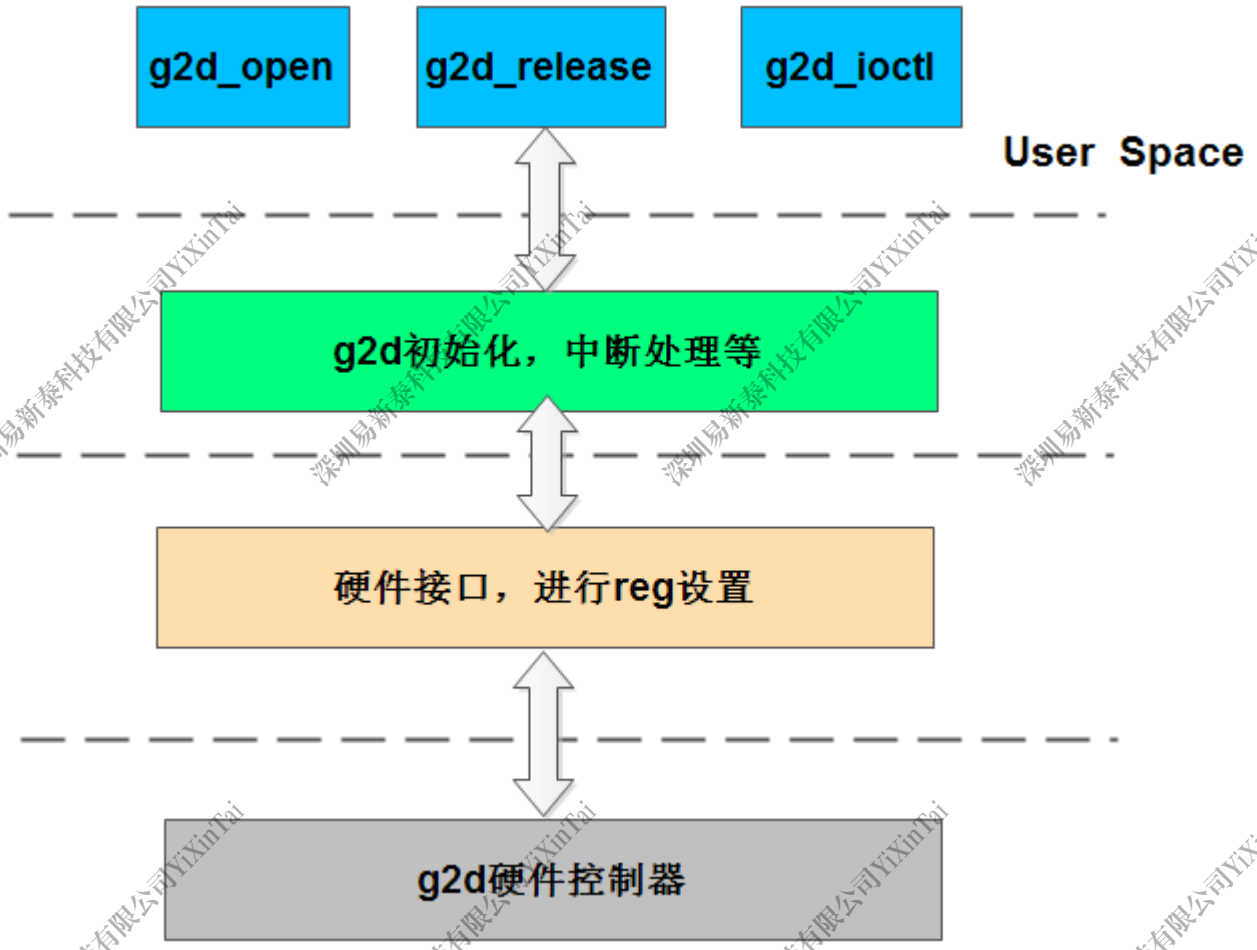
版权所有 © 珠海全志科技股份有限公司。保留一切权利

第 3 章 G2D 模块驱动开发

3.1 G2D 驱动框架

G2D 驱动框架如下图所示：

图 3-1: G2D 驱动框架



G2D 驱动加载后对模块驱动结构数据进行相关初始化操作，注册中断，创建/dev/g2d 节点，应用程序 open 打开此节点，通过 ioctl 下达 g2d 操作命令并传递相关参数，g2d 中间抽象层解析参数，并调用各个子模块的底层硬件寄存器操作接口，配置寄存器，启动硬件。完成一次 g2d 任务后触发中断，通知驱动任务完成，并将操作结果返回到用户。

3.2 源码目录

3.2.1 g2d_rcq 目录（适配 g2d1XX IP）

g2d_rcq 驱动的主要源码文件及说明如下：

```
.
├── g2d_bld.c               //blend子模块底层硬件相关
├── g2d_bld.h
├── g2d_bsp.h
├── g2d.c                   //g2d驱动顶层实现
├── g2d_debug.c             //g2d驱动调试节点相关实现
├── g2d_debug.h
├── g2d_driver.h        //对接应用层的头文件
├── g2d_mixer.c             //g2d操作中间抽象层
├── g2d_mixer.h
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

```
├── g2d_mixer_type.h        //mixer模块寄存器定义，包含bld、scaler、ovl_v、ovl_u、wb子模块寄存器定义
├── g2d_ovl_u.c             //ui通道底层硬件相关
├── g2d_ovl_u.h
├── g2d_ovl_v.c             //video通道底层硬件相关
├── g2d_ovl_v.h
├── g2d_rcq.c               //rcq buffer内存申请接口定义
├── g2d_rcq.h
├── g2d_rotate.c            //rotate子模块底层硬件相关
├── g2d_rotate.h
├── g2d_rotate_type.h       //rotate子模块寄存器定义
├── g2d_scal.c              //scaler子模块底层硬件相关
├── g2d_scal.h
├── g2d_top.c               //top寄存器底层硬件相关
├── g2d_top.h
├── g2d_top_type.h          //top寄存器定义
├── g2d_wb.c                //write back子模块底层硬件相关
├── g2d_wb.h
└── Makefile
```

3.2.2 g2d200 目录（适配 g2d2XX IP）

g2d 200 版本代码位于 bsp/drivers/g2d/g2d200 目录下。

源码目录如下：

```
.
├── g2d_bld.c           //blend子模块底层硬件相关
├── g2d_bld.h
├── g2d_bsp.h
├── g2d.c               //g2d驱动顶层实现
├── g2d_core.c          //core寄存器硬件操作相关
├── g2d_core.h
├── g2d_debug.c         //g2d驱动调试节点相关实现
├── g2d_debug.h
├── g2d_driver_i.h
├── g2d_mixer.c         //g2d操作中间抽象层
├── g2d_mixer.h
├── g2d_driver.h        //对接应用层的头文件
├── g2d_mixer_type.h    //mixer模块寄存器定义，包含bld、scaler、ovl_v、ovl_u、wb子模块寄存器定义
├── g2d_ovl_u.c         //ui通道底层硬件操作相关
├── g2d_ovl_u.h
├── g2d_ovl_v.c         //video通道底层硬件相关
├── g2d_ovl_v.h
├── g2d_platform.c      //g2d操作中间抽象层
├── g2d_platform.h
├── g2d_rcq.c           //rcq buffer内存申请接口定义
├── g2d_rcq.h
├── g2d_rotate.c        //rotate子模块底层硬件相关
├── g2d_rotate.h
├── g2d_rotate_type.h   //rotate子模块寄存器定义
├── g2d_scal.c          //scaler子模块底层硬件相关
├── g2d_scal.h
├── g2d_thread.c        //g2d线程操作接口定义
├── g2d_thread.h
├── g2d_thread_type.h   //g2d线程相关寄存器定义
├── g2d_top.c           //top寄存器底层硬件相关
├── g2d_top.h
├── g2d_top_type.h      //top寄存器定义
├── g2d_wb.c            //write back子模块底层硬件相关
├── g2d_wb.h
└── Makefile
```

3.3 配置方式

G2D 模块需要配置的参数主要包含 G2D 寄存器起始地址、中断号。举例，V883 平台，上述配置参数可在 rtos-hal/hal/source/g2d_rcq/g2d.c 中相应的配置：

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

```
#define SUNXI_GIC_START 32
#define SUNXI_IRQ_G2D (SUNXI_GIC_START + 21)
#define SUNXI_G2D_START 0x01480000
```

F102 平台，上述配置参数可在 rtos-hal/hal/source/g2d_200/g2d.c 中相应的配置：

```
...
#elif defined(CONFIG_ARCH_SUN252IW2)
#if defined(CONFIG_ARCH_RISCV)
#define SUNXI_IRQ_G2D1              111             /* 配置中断号 */
#define SUNXI_IRQ_G2D2              112
#define SUNXI_IRQ_G2D3              113
#define SUNXI_IRQ_G2D4              114
#endif

#define SUNXI_G2D_START             0x05410000      /* 寄存器起始地址 */

#define SUNXI_G2D_CLK_PARENT        CLK_PLL_PERI_2X /* 时钟配置 */
#define SUNXI_G2D_CLK_ID            CLK_G2D
#define SUNXI_G2D_RESET_ID          RST_BUS_G2D
#define SUNXI_G2D_CLK_BUS_ID        CLK_BUS_G2D
#define SUNXI_G2D_CLK_MBUS_ID       CLK_BUS_G2D_M
#define SUNXI_G2D_CLK_RATE          300000000       /* 时钟频率 */
#define HAL_SUNXI_RESET             AW_SYS_CCU
#define HAL_SUNXI_CCU               AW_SYS_CCU

#elif
...
```

3.4 测试用例

make menuconfig 打开 HAL_TEST_G2D 配置。

测试用例位于 melis4.0/lichee/rtos-hal/hal/test/g2d 路径下。

测试用例	功能
test_g2d_bitblt.c	测试 g2d bitblt 单输入单输出操作：旋转镜像、缩放、裁剪拼接、矩形填充、图层混合（dst 和 result 共用内存）、二元光栅（dst 和 result 共用内存）
test_g2d_blend2.c	测试图层混合（src0、src1、dst）、二元光栅（src0、src1、dst）、三元光栅
g2d_mixer_perf.c	g2d rotate 和 mixer 功能性能测试（FPS）

测试用例命令参数参考对应项目测试报告。

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

第 4 章 数据结构

4.1 g2d_blt_flags

- DESCRIPTION g2d_blt_flags 用于描述一个 bitblt 和 stretchblt 的 flag 属性信息
- PROTOTYPE

```
typedef enum {
    G2D_BLT_NONE               = 0x00000000,
    G2D_BLT_PIXEL_ALPHA     = 0x00000001,
    G2D_BLT_PLANE_ALPHA     = 0x00000002,
    G2D_BLT_MULTI_ALPHA     = 0x00000004,
    G2D_BLT_SRC_COLORKEY    = 0x00000008,
    G2D_BLT_DST_COLORKEY    = 0x00000010,
    G2D_BLT_FLIP_HORIZONTAL = 0x00000020,
    G2D_BLT_FLIP_VERTICAL   = 0x00000040,
    G2D_BLT_ROTATE90           = 0x00000080,
    G2D_BLT_ROTATE180       = 0x00000100,
    G2D_BLT_ROTATE270       = 0x00000200,
    G2D_BLT_MIRROR45           = 0x00000400,
    G2D_BLT_MIRROR135       = 0x00000800,
}g2d_blt_flags;
```

- MEMBERS

```
G2D_BLT_NONE          - 纯拷贝
G2D_BLT_PIXEL_ALPHA   - 点alpha标志
G2D_BLT_PLANE_ALPHA     - 面alpha标志
G2D_BLT_MULTI_ALPHA     - 混合alpha标志
G2D_BLT_SRC_COLORKEY    - 源colorkey标志
G2D_BLT_DST_COLORKEY    - 目标colorkey标志
G2D_BLT_FLIP_HORIZONTAL - 水平翻转
G2D_BLT_FLIP_VERTICAL    - 垂直翻转
G2D_BLT_ROTATE90             - 逆时针旋转90度
G2D_BLT_ROTATE180          - 逆时针旋转180度
G2D_BLT_ROTATE270          - 逆时针旋转270度
G2D_BLT_MIRROR45             - 镜像45度
G2D_BLT_MIRROR135          - 镜像135度
```

4.2 g2d_fillrect_flags

- DESCRIPTION g2d_fillrect_flags 用于描述一个 fillrect 属性信息
- PROTOTYPE

```
typedef enum {
    G2D_FIL_NONE              = 0x00000000,
    G2D_FIL_PIXEL_ALPHA     = 0x00000001,
    G2D_FIL_PLANE_ALPHA     = 0x00000002,
    G2D_FIL_MULTI_ALPHA     = 0x00000004,
}g2d_fillrect_flags;
```

- MEMBERS

```
G2D_FIL_NONE        - 纯填充
G2D_FIL_PIXEL_ALPHA - 填充区域和目标做点alpha
G2D_FIL_PLANE_ALPHA - 填充区域和目标做面alpha
G2D_FIL_MULTI_ALPHA - 填充区域的alpha值*面alpha值后再和目标做alpha
```

深圳易新泰科技有限公司YiXinTai

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4.3 g2d_data_fmt(version 1.0)

- DESCRIPTION
  g2d_data_fmt 用于描述像素格式
- PROTOTYPE
  1.0 版本支持的图像格式

```
typedef enum {
  G2D_FMT_ARGB_AYUV8888 = (0x0),
    G2D_FMT_BGRA_VUYA8888   = (0x1),
    G2D_FMT_ABGR_AVUY8888   = (0x2),
    G2D_FMT_RGBA_YUVA8888   = (0x3),
    G2D_FMT_XRGB8888            = (0x4),
    G2D_FMT_BGRX8888            = (0x5),
    G2D_FMT_XBGR8888            = (0x6),
    G2D_FMT_RGBX8888            = (0x7),
    G2D_FMT_ARGB4444            = (0x8),
    G2D_FMT_ABGR4444            = (0x9),
    G2D_FMT_RGBA4444            = (0xA),
    G2D_FMT_BGRA4444            = (0xB),
    G2D_FMT_ARGB1555            = (0xC),
    G2D_FMT_ABGR1555            = (0xD),
    G2D_FMT_RGBA5551            = (0xE),
    G2D_FMT_BGRA5551            = (0xF),
    G2D_FMT_RGB565          = (0x10),
    G2D_FMT_BGR565          = (0x11),
    G2D_FMT_IYUV422         = (0x12),
    G2D_FMT_8BPP_MONO       = (0x13),
    G2D_FMT_4BPP_MONO       = (0x14),
    G2D_FMT_2BPP_MONO       = (0x15),
    G2D_FMT_1BPP_MONO       = (0x16),
    G2D_FMT_PYUV422UVC      = (0x17),
    G2D_FMT_PYUV420UVC      = (0x18),
    G2D_FMT_PYUV411UVC      = (0x19),

只有输出才有的格式:
    G2D_FMT_PYUV422         = (0x1A),
    G2D_FMT_PYUV420         = (0x1B),
    G2D_FMT_PYUV411         = (0x1C),

只有输入才支持的格式:
    G2D_FMT_8BPP_PALETTE    = (0x1D),
    G2D_FMT_4BPP_PALETTE    = (0x1E),
    G2D_FMT_2BPP_PALETTE    = (0x1F),
    G2D_FMT_1BPP_PALETTE    = (0x20),
    G2D_FMT_PYUV422UVC_MB16 = (0x21),
    G2D_FMT_PYUV420UVC_MB16 = (0x22),
    G2D_FMT_PYUV411UVC_MB16 = (0x23),
    G2D_FMT_PYUV422UVC_MB32 = (0x24),
    G2D_FMT_PYUV420UVC_MB32 = (0x25),
    G2D_FMT_PYUV411UVC_MB32 = (0x26),
    G2D_FMT_PYUV422UVC_MB64 = (0x27),
    G2D_FMT_PYUV420UVC_MB64 = (0x28),
    G2D_FMT_PYUV411UVC_MB64 = (0x29),
    G2D_FMT_PYUV422UVC_MB128= (0x2A),
    G2D_FMT_PYUV420UVC_MB128= (0x2B),
    G2D_FMT_PYUV411UVC_MB128= (0x2C),
}g2d_data_fmt;
```

- MEMBERS

```
G2D_FMT_ARGB8888        : alpha(8bit)R(8bit)G(8bit)B(8bit)
G2D_FMT_BGRA8888        : B(8bit)G(8bit)R(8bit)alpha(8bit)
G2D_FMT_ABGR8888        : alpha(8bit)B(8bit)G(8bit)R(8bit)
G2D_FMT_RGBA8888        : R(8bit)G(8bit)B(8bit)alpha(8bit)

G2D_FMT_XRGB8888        : 24bit,RGB各8bit,alpha为高位自动填充为0xFF
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

文档问题反馈: aw-document@allwinnertech.com

```
G2D_FMT_BGRX8888        : 24bit,BGR各8bit,alpha为低位自动填充为0xFF
G2D_FMT_XBGR8888        : 24bit,BGR各8bit,alpha为高位自动填充为0xFF
G2D_FMT_RGBX8888        : 24bit,RGB各8bit,alpha为低位自动填充为0xFF

G2D_FMT_ARGB4444        : alpha(4bit)R(4bit)G(4bit)B(4bit)
G2D_FMT_BGRA4444        : B(4bit)G(4bit)R(4bit)alpha(4bit)
G2D_FMT_ABGR4444        : alpha(4bit)B(4bit)G(4bit)R(4bit)
G2D_FMT_RGBA4444        : R(4bit)G(4bit)B(4bit)alpha(4bit)
G2D_FMT_ARGB1555        : alpha(1bit)R(5bit)G(5bit)B(5bit)
G2D_FMT_BGRA1555        : B(5bit)G(5bit)R(5bit)alpha(1bit)
G2D_FMT_ABGR1555        : alpha(1bit)B(5bit)G(5bit)R(5bit)
G2D_FMT_RGBA1555        : R(5bit)G(5bit)B(5bit)alpha(1bit)

G2D_FMT_RGB565      : R(5bit)G(6bit)B(5bit)
G2D_FMT_BGR565      : B(5bit)G(6bit)R(5bit)

G2D_FMT_IYUV422     : Interleaved YUV422

G2D_FMT_8BPP_MONO   : 8bit per pixel mono
G2D_FMT_4BPP_MONO   : 4bit per pixel mono
G2D_FMT_2BPP_MONO   : 2bit per pixel mono
G2D_FMT_1BPP_MONO   : 1bit per pixel mono

G2D_FMT_PYUV422UVC  : Planar UV combined only
G2D_FMT_PYUV420UVC  : Planar UV combined only
G2D_FMT_PYUV411UVC  : Planar UV combined only

G2D_FMT_PYUV422     : Planar YUV422
G2D_FMT_PYUV420     : Planar YUV420
G2D_FMT_PYUV411     : Planar YUV411

G2D_FMT_8BPP_PALETTE: 8bit per pixel palette only for input
G2D_FMT_4BPP_PALETTE: 4bit per pixel palette only for input
G2D_FMT_2BPP_PALETTE: 2bit per pixel palette only for input
G2D_FMT_1BPP_PALETTE: 1bit per pixel palette only for input

G2D_FMT_PYUV422UVC_MB16: 16x16 tile base planar uv combined only for input
G2D_FMT_PYUV420UVC_MB16: 16x16 tile base planar uv combined only for input
G2D_FMT_PYUV411UVC_MB16: 16x16 tile base planar uv combined only for input
G2D_FMT_PYUV422UVC_MB32: 16x16 tile base planar uv combined only for input
G2D_FMT_PYUV420UVC_MB32: 16x16 tile base planar uv combined only for input
G2D_FMT_PYUV411UVC_MB32: 16x16 tile base planar uv combined only for input
G2D_FMT_PYUV422UVC_MB64: 16x16 tile base planar uv combined only for input
G2D_FMT_PYUV420UVC_MB64: 16x16 tile base planar uv combined only for input
G2D_FMT_PYUV411UVC_MB64: 16x16 tile base planar uv combined only for input
G2D_FMT_PYUV422UVC_MB128: 16x16 tile base planar uv combined only for input
G2D_FMT_PYUV420UVC_MB128: 16x16 tile base planar uv combined only for input
G2D_FMT_PYUV411UVC_MB128: 16x16 tile base planar uv combined only for input
```

4.4 g2d_pixel_seq(version 1.0)

- DESCRIPTION g2d_pixel_seq 用于描述像素序列
- PROTOTYPE

```
typedef enum {
    G2D_SEQ_NORMAL                  = 0x0,
    G2D_SEQ_VYUY                    = 0x1,
    G2D_SEQ_YVYU                    = 0x2,
    G2D_SEQ_VUVU                    = 0x3,
    G2D_SEQ_P10                     = 0x4,
    G2D_SEQ_P01                     = 0x5,
    G2D_SEQ_P3210                   = 0x6,
    G2D_SEQ_P0123                   = 0x7,
    G2D_SEQ_P76543210           = 0x8,
    G2D_SEQ_P67452301           = 0x9,
    G2D_SEQ_P10325476           = 0xA,
    G2D_SEQ_P01234567           = 0xB,
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

```
    G2D_SEQ_2BPP_BIG_BIG        = 0xC,
    G2D_SEQ_2BPP_BIG_LITTER      = 0xD,
    G2D_SEQ_2BPP_LITTER_BIG      = 0xE,
    G2D_SEQ_2BPP_LITTER_LITTER   = 0xF,
    G2D_SEQ_1BPP_BIG_BIG           = 0x10,
    G2D_SEQ_1BPP_BIG_LITTER        = 0x11,
    G2D_SEQ_1BPP_LITTER_BIG        = 0x12,
    G2D_SEQ_1BPP_LITTER_LITTER   = 0x13,

  } g2d_pixel_seq;
```

- MEMBERS

```
G2D_SEQ_NORMAL          : Normal sequence

//for interleaved yuv422
G2D_SEQ_VYUY                : pixel 0在低16位
G2D_SEQ_YVYU                : pixel 1在低16位

// for uv_combined yuv420
G2D_SEQ_VUVU                : Planar VU combined only

// for 16bpp rgb
G2D_SEQ_P10             : pixel 0在低16位
G2D_SEQ_P01             : pixel 1在低16位

// planar format or 8bpp rgb
G2D_SEQ_P3210               : pixel 0在低8位
G2D_SEQ_P0123               : pixel 3在低8位

// for 4bpp rgb
G2D_SEQ_P76543210       :   7,6,5,4,3,2,1,0
G2D_SEQ_P67452301       :   6,7,4,5,2,3,0,1
G2D_SEQ_P10325476       :   1,0,3,2,5,4,7,6
G2D_SEQ_P01234567       :   0,1,2,3,4,5,6,7

// for 2bpp rgb
G2D_SEQ_2BPP_BIG_BIG        :
15,14,13,12,11,10,9,8,7,6,5,4,3,2,1,0

G2D_SEQ_2BPP_BIG_LITTER     :
12,13,14,15,8,9,10,11,4,5,6,7,0,1,2,3

G2D_SEQ_2BPP_LITTER_BIG     :
3,2,1,0,7,6,5,4,11,10,9,8,15,14,13,12

G2D_SEQ_2BPP_LITTER_LITTER  :
0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15

// for 1bpp rgb
G2D_SEQ_1BPP_BIG_BIG        :
31,30,29,28,27,26,25,24,23,22,21,20,19,18,17,16,15,14,13,12,11,10,9,8,7,6,5,4,3,2,1,0

G2D_SEQ_1BPP_BIG_LITTER     :
24,25,26,27,28,29,30,31,16,17,18,19,20,21,22,23,8,9,10,11,12,13,14,15,0,1,2,3,4,5,6,7

G2D_SEQ_1BPP_LITTER_BIG     :
7,6,5,4,3,2,1,0,15,14,13,12,11,10,9,8,23,22,21,20,19,18,17,16,31,30,29,28,27,26,25,24

G2D_SEQ_1BPP_LITTER_LITTER      :
0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,22,23,24,25,26,27,28,29,30,31
```

4.5 g2d_blt_flags_h

- DESCRIPTION
  g2d_blt_flags_h 定义二元光栅操作码
- PROTOTYPE

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

```
typedef enum {
    G2D_BLT_NONE_0 = 0x0,
    G2D_BLT_BLACKNESS,
    G2D_BLT_NOTMERGEPEN,
    G2D_BLT_MASKNOTPEN,
    G2D_BLT_NOTCOPYPEN,
    G2D_BLT_MASKPENNOT,
    G2D_BLT_NOT,
    G2D_BLT_XORPEN,
    G2D_BLT_NOTMASKPEN,
    G2D_BLT_MASKPEN,
    G2D_BLT_NOTXORPEN,
    G2D_BLT_NOP,
    G2D_BLT_MERGENOTPEN,
    G2D_BLT_COPYPEN,
    G2D_BLT_MERGEPENNOT,
    G2D_BLT_MERGEPEN,
    G2D_BLT_WHITENESS = 0x000000ff,

    G2D_ROT_90   =    0x00000100,
    G2D_ROT_180  =    0x00000200,
    G2D_ROT_270  =    0x00000300,
    G2D_ROT_H    =    0x00001000,
    G2D_ROT_V    =    0x00002000,

//G2D_SM_TDLR_1  =    0x10000000,
    G2D_SM_DTLR_1  =    0x10000000,
//G2D_SM_TDRL_1  =    0x20000000,
//G2D_SM_DTRL_1  =    0x30000000,
} g2d_blt_flags_h;
```

- MEMBERS

```
MEMBER  DESCRIPTION
G2D_BLT_NONE    单个源操作
G2D_BLT_BLACK   BLACKNESS               :使用与物理调色板的索引0相关的色彩来填充目标矩形区域,(对缺省的物理调色板,该颜色为黑色)
G2D_BLT_NOTMERGEPEN dst = ~(dst+src)  :
G2D_BLT_MASKNOTPEN  dst =~src&dst
G2D_BLT_NOTCOPYPEN  dst =~src
G2D_BLT_MASKPENNOT  dst =src&~dst
G2D_BLT_NOT dst =~dst                   :使目标矩形区域颜色取反
G2D_BLT_XORPEN  dst =src^dst
G2D_BLT_NOTMASKPEN  dst =~(src&dst)
G2D_BLT_MASKPEN dst =src&dst
G2D_BLT_NOTXORPEN   dst =~(src^dst)
G2D_BLT_NOP dst =dst
G2D_BLT_MERGENOTPEN dst =~src+dst
G2D_BLT_COPEPEN dst =src
G2D_BLT_MERGEPENNOT dst =src+~dst
G2D_BLT_MERGEPEN    dst =src+dst
G2D_BLT_WHITE   WHITENESS               :使用与物理调色板中索引1有关的颜色填充目标矩形区域(对于缺省物理调色板来说,这个颜色为白色)
```

4.6 g2d_image(version 1.0)

- DESCRIPTION
  g2d_image 用于描述 image 属性信息
- PROTOTYPE

```
typedef struct {
  __u32          addr[3];
  __u32          w;
  __u32          h;
  g2d_data_fmt   format;
  g2d_pixel_seq  pixel_seq;
}g2d_image;
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

- MEMBERS

```
addr[3]:   图像帧的基地址，对于UV combined，addr[0,1]有效，planar类型addr[0,1，2]有效，其他addr[0]有效
w:         图像帧的宽
h:         图像帧的高
format:    图像帧buffer的像素格式，详见g2d_data_fmt
pixel_seq: 图像帧buffer的像素序列，详见g2d_pixel_seq
```

4.7 g2d_image_enh

- DESCRIPTION
  g2d_image_enh 主要描述图片的宽高、存放地址、是否做 Clip 处理，是否为预乘等。
- PROTOTYPE

```
typedef struct {
  int           bbuff;
    __u32       color;
    g2d_fmt_enh format;
    __u32           laddr[3];
    __u32           haddr[3];
    __u32           width;
    __u32           height;
    __u32       align[3];
    g2d_rect    clip_rect;
    __u32         gamut;
    int             bpremul;
    __u8            alpha;
    g2d_alpha_mode_enh      mode;
    int fd;
    u32 use_phy_addr;
} g2d_image_enh;
```

- MEMBERS

```
MEMBER  DESCRIPTION
format:         图格式
laddr   Buffer: 起始低位地址
haddr   Buffer: 起始高位地址
width       : 图宽度（in pixel）
height      :   图高度（in pixel）
pitch       : Buffer的pitch
clip_rect   :   ROI矩形
gamut       : 图的色域
bpremul     : 是否为预乘
alpha       : 面alpha值
mode        : alpha模式设置
use_phy_addr: 是否使用物理地址的标志。1表示使用，0表示使用fd
```

注意: 当 use_phy_addr 为 1 的时候，你必须自己设置好 laddr 和 haddr，并自行做好偏移。使用 fd 则不需要设置这两个数组。

4.8 g2d_fmt_enh

- DESCRIPTION
  g2d_fmt_enh 用于描述 G2D 模块支持的格式
- PROTOTYPE

```
typedef enum{
    G2D_FORMAT_ARGB8888,
    G2D_FORMAT_ABGR8888,
    G2D_FORMAT_RGBA8888,
    G2D_FORMAT_BGRA8888,
    G2D_FORMAT_XRGB8888,
    G2D_FORMAT_XBGR8888,
    G2D_FORMAT_RGBX8888,
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

```
    G2D_FORMAT_BGRX8888,
    G2D_FORMAT_RGB888,
    G2D_FORMAT_BGR888,
    G2D_FORMAT_RGB565,
    G2D_FORMAT_BGR565,
    G2D_FORMAT_ARGB4444,
    G2D_FORMAT_ABGR4444,
    G2D_FORMAT_RGBA4444,
    G2D_FORMAT_BGRA4444,
    G2D_FORMAT_ARGB1555,
    G2D_FORMAT_ABGR1555,
    G2D_FORMAT_RGBA5551,
    G2D_FORMAT_BGRA5551,
    G2D_FORMAT_ARGB2101010,
    G2D_FORMAT_ABGR2101010,
    G2D_FORMAT_RGBA1010102,
    G2D_FORMAT_BGRA1010102,

    /* invailed for UI channel */
    G2D_FORMAT_IYUV422_V0Y1U0Y0 = 0x20,
    G2D_FORMAT_IYUV422_Y1V0Y0U0,
    G2D_FORMAT_IYUV422_U0Y1V0Y0,
    G2D_FORMAT_IYUV422_Y1U0Y0V0,

    G2D_FORMAT_YUV422UVC_V1U1V0U0,
    G2D_FORMAT_YUV422UVC_U1V1U0V0,
    G2D_FORMAT_YUV422_PLANAR,

    G2D_FORMAT_YUV420UVC_V1U1V0U0 = 0x28,
    G2D_FORMAT_YUV420UVC_U1V1U0V0,
    G2D_FORMAT_YUV420_PLANAR,

    G2D_FORMAT_YUV411UVC_V1U1V0U0 = 0x2c,
    G2D_FORMAT_YUV411UVC_U1V1U0V0,
    G2D_FORMAT_YUV411_PLANAR,

    G2D_FORMAT_Y8 = 0x30,

    /* YUV 10bit format */
    G2D_FORMAT_YVU10_P010 = 0x34,

    G2D_FORMAT_YVU10_P210 = 0x36,

    G2D_FORMAT_YVU10_444 = 0x38,
    G2D_FORMAT_YUV10_444 = 0x39,
}g2d_fmt_enh;
```

4.9 g2d_rop3_cmd_flag

- DESCRIPTION
  g2d_rop3_cmd_flag 用于定义三元光栅操作码
- PROTOTYPE

```
typedef enum {
    G2D_ROP3_BLACKNESS   = 0x00,
    G2D_ROP3_NOTSRCERASE = 0x11,
    G2D_ROP3_NOTSRCCOPY  = 0x33,
    G2D_ROP3_SRCERASE    = 0x44,
    G2D_ROP3_DSTINVERT   = 0x55,
    G2D_ROP3_PATINVERT   = 0x5A,
    G2D_ROP3_SRCINVERT   = 0x66,
    G2D_ROP3_SRCAND      = 0x88,
    G2D_ROP3_MERGEPAINT  = 0xBB,
    G2D_ROP3_MERGECOPY   = 0xC0,
    G2D_ROP3_SRCCOPY     = 0xCC,
    G2D_ROP3_SRCPAINT    = 0xEE,
    G2D_ROP3_PATCOPY     = 0xF0,
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

```
    G2D_ROP3_PATPAINT    = 0xFB,
    G2D_ROP3_WHITENESS   = 0xFF,
}g2d_rop3_cmd_flag;
```

- MEMBERS

```
MEMBER  DESCRIPTION
G2D_ROP3_BLACKNESS    dst = BLACK
G2D_ROP3_NOTSRCERASE    dst = (NOT src) AND (NOT dst)
G2D_ROP3_NOTSRCCOPY     dst = (NOT src)        :将源矩形区域颜色取反,拷贝到目标矩形区域
G2D_ROP3_SRCERASE       dst = src AND (NOT dst )
G2D_ROP3_DSTINVERT    dst = (NOT dst)
G2D_ROP3_PATINVERT    dst = pattern XOR dst    :通过使用布尔型的异或(XOR)操作符将特定模式和目标矩形区域颜色合并
G2D_ROP3_SRCINVERT    dst = src XOR dst        :通过使用布尔型的异或(XOR)操作符将源和目标矩形区域颜色合并
G2D_ROP3_SRCAND       dst = srcAND dst         :通过使用与操作符将源和目标矩形区域颜色值合并
G2D_ROP3_MERGEPAINT   dst = (NOT src) OR dst   :通过使用布尔型的或(OR)操作符将反向的源矩形区域的颜色与目标矩形区域颜色合并
G2D_ROP3_MERGECOPY    dst = (src AND pattern)
G2D_ROP3_SRCCOPY        dst = src                :将源矩形区域直接拷贝到目标矩形区域
G2D_ROP3_SRCPAINT       dst = src OR dst         :通过使用布尔型的或(OR)操作符将源和目标矩形区域颜色合并
G2D_ROP3_PATCOPY        dst = pattern
G2D_ROP3_PATPAINT       dst = DPSnoo             :通过使用布尔型的或(OR)操作符将源矩形区域取反后的颜色值与特定模式的颜色合并,然后使用OR
     操作符与该操作的结果与目标矩形区域内的颜色合并.
G2D_ROP3_WHITENESS    dst = WHITE
```

4.10 g2d_bld_cmd_flag

- DESCRIPTION
  g2d_bld_cmd_flag 定义 BLD 操作命令
- PROTOTYPE

```
typedef enum {
    G2D_BLD_CLEAR     = 0x00000001,
    G2D_BLD_COPY      = 0x00000002,
    G2D_BLD_DST       = 0x00000003,
    G2D_BLD_SRCOVER   = 0x00000004,
    G2D_BLD_DSTOVER   = 0x00000005,
    G2D_BLD_SRCIN     = 0x00000006,
    G2D_BLD_DSTIN     = 0x00000007,
    G2D_BLD_SRCOUT    = 0x00000008,
    G2D_BLD_DSTOUT    = 0x00000009,
    G2D_BLD_SRCATOP   = 0x0000000a,
    G2D_BLD_DSTATOP   = 0x0000000b,
    G2D_BLD_XOR       = 0x0000000c,
    G2D_CK_SRC        = 0x00010000,
    G2D_CK_DST        = 0x00020000,
}g2d_bld_cmd_flag;
```

4.11 g2d_ck

- DESCRIPTION
  g2d_ck 定义了 colorkey 操作的参数
- PROTOTYPE

```
typedef struct {
    int match_rule;
    __u32 max_color;
    __u32 min_color;
}g2d_ck;
```

- MEMBERS

深圳易新泰科技有限公司YiXinTai

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
MEMBER  DESCRIPTION
match_rule  当match_rule为假时，Color Min=<Color<=Color Max表示满足匹配条件
当match_rule为真时，Color>Color Max or Color <Color Min表示满足匹配条件
ck_max_color    Color Max
ck_min_color    Color Min
```

4.12 g2d_alpha_mode_enh

- DESCRIPTION
  g2d_alpha_mode_enh 定义进行 alpha blend 操作时，选择的 alpha mode
- PROTOTYPE

```
typedef enum{
    G2D_PIXEL_ALPHA,
    G2D_GLOBAL_ALPHA,
    G2D_MIXER_ALPHA,
}g2d_alpha_mode_enh;
```

- MEMBERS

```
MEMBER  DESCRIPTION
G2D_PIXEL_ALPHA 点alpha
G2D_GLOBAL_ALPHA    面alpha
G2D_MIXER_ALPHA 混合alpha
```

4.13 g2d_color_gmt

- DESCRIPTION
  g2d_color_gmt 定义进行位操作时，选择的颜色空间
- PROTOTYPE

```
typedef enum{
    G2D_BT601,
    G2D_BT709,
    G2D_BT2020,
}g2d_color_gmt;
```

4.14 g2d_scan_order(version 1.0)

- DESCRIPTION
  g2d_scan_order 定义进行 alpha blend 操作时，选择的图像扫行模式
- PROTOTYPE

```
enum g2d_scan_order {
    G2D_SM_TDLR = 0x00000000,
    G2D_SM_TDRL = 0x00000001,
    G2D_SM_DTLR = 0x00000002,
    G2D_SM_DTRL = 0x00000003,
};
```

- MEMBERS

```
MEMBER  DESCRIPTION
G2D_SM_TDLR Top to down, Left to right
G2D_SM_DTLR Down to top, Left to right
G2D_SM_TDRL Top to down, Right to left
G2D_SM_DTRL Down to top, Left to right
```

深圳易新泰科技有限公司YiXinTai

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4.15 g2d_blt(version 1.0)

- DESCRIPTION
  g2d_blt 用于一个源和目标做 blt 的信息
- PROTOTYPE

```
typedef struct {
    g2d_blt_flags        flag;
    g2d_image            src_image;
    g2d_rect             src_rect;
    g2d_image            dst_image;
    __s32                dst_x;
    __s32                dst_y;
    __u32                color;
    __u32                alpha;
}g2d_blt;
```

- MEMBERS

```
flag        : block transfer标志，详见g2d_blt_flags
src_image   : 源图像信息，详见g2d_image
dst_image   : 目标图像信息，详见g2d_image
dst_x       : 目标矩形左上角x
dst_y       : 目标矩形左上角y
color       : colorkey颜色
alpha       : 面alpha值
```

4.16 g2d_fillrect(version 1.0)

- DESCRIPTION
  g2d_fillrect 用于描述一个 fill rectangle 参数信息
- PROTOTYPE

```
typedef struct {
    g2d_fillrect_flags   flag;
    g2d_image            dst_image;
    g2d_rect             dst_rect;
    __u32                color;
    __u32                alpha;
}g2d_fillrect;
```

- MEMBERS

```
flag        : 填充矩形标志，详见g2d_fillrect_flags
dst_image   : 目标图像信息，详见g2d_image
dst_rect    : 目标矩形信息，x/y/w/h-左上角x/左上角y/宽/高
color       : 填充颜色
alpha       : 面alpha值
```

4.17 g2d_stretchblt(version 1.0)

- DESCRIPTION g2d_stretchblt 用于描述一个 stretchblt 参数信息
- PROTOTYPE

```
typedef struct {
    g2d_blt_flags        flag;
    g2d_image            src_image;
    g2d_rect             src_rect;
    g2d_image            dst_image;
    g2d_rect             dst_rect;
    __u32                color;
    __u32                alpha;
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

```
} g2d_stretchblt;
```

- MEMBERS

```
flag        : block transfer标志，详见g2d_blt_flags
src_image   : 源图像信息，详见g2d_image
src_rect    : 源矩形信息，x/y/w/h-左上角x/左上角y/宽/高
dst_image   : 目标图像信息，详见g2d_image
dst_rect    : 目标矩形信息，x/y/w/h-左上角x/左上角y/宽/高
color       : colorkey颜色
alpha       : 面alpha值
```

4.18 g2d_blt_h

- DESCRIPTION
  g2d_blt_h 实现对 foreground 带缩放的 ROP2 处理。
- PROTOTYPE

```
typedef struct {
    g2d_blt_flags_h      flag_h;
    g2d_image_enh        src_image_h;
    g2d_image_enh        dst_image_h;
    __u32                    color;
    __u32                    alpha;
}g2d_blt_h;
```

- MEMBERS

```
flag_h          : blt操作flag标志，增强版标志
src_image_h   : 源图像信息,增强版的图像参数,详见g2d_image_enh
dst_image_h   : 目标图像信息，增强版的图像参数
color             : colorkey颜色
alpha             : 面alpha值
```

4.19 g2d_bld(version 1.0)

- DESCRIPTION
  g2d_bld 实现两幅图的 BLD 和 colorkey 操作。
- PROTOTYPE

```
typedef struct {
    g2d_bld_cmd_flag     bld_cmd;
    g2d_image_enh        dst_image_h;
    g2d_image_enh        src_image_h;
    g2d_ck               ck_para;
}g2d_bld;/* blending enhance */
```

- MEMBERS

```
bld_cmd     : blending的操作flag标志，增强版标志
src_image_h : 源图像信息,增强版的图像参数
dst_image_h : 目标图像信息，增强版的图像参数
ck_para     : colorkey参数
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

第 5 章 G2D 模块接口和使用 Demo

本章介绍 G2D 模块接口和功能使用 Demo，模块应用层接口头文件源码参考驱动目录下的 g2d_driver.h 头文件。首先介绍各个 g2d 单独功能的相关参数、接口使用，接着介绍用于提升调用效率的批处理参数工作模式，最后介绍各个独立功能并行处理的混合操作。

G2D 驱动向 OS 或其他 driver hal 暴露的接口如下表，模块使用主要通过 ioctl 实现，即 sunxi_g2d_control，后续将主要介绍该接口的用法。

表 5-1: API 说明

API	解释说明
g2d_probe	初始化 g2d 驱动
sunxi_g2d_control	g2d 驱动 ioctl 接口
sunxi_g2d_close	关闭 g2d 驱动
sunxi_g2d_open	打开 g2d 驱动

5.1 旋转和镜像 (Rotate/Mirror)

5.1.1 关键参数配置

以下是旋转和镜像功能的关键结构体和参数。

g2d_blt_h 是单幅图片（bitblt）操作的参数结构体

```
typedef struct {
    g2d_blt_flags_h flag_h;
    g2d_image_enh src_image_h;
    g2d_image_enh dst_image_h;
} g2d_blt_h;
```

g2d_blt_flags_h 是操作枚举，其中从 G2D_ROT_90 到 G2D_ROT_90_V 是旋转相关的操作枚举，用于配置 g2d_blt_h 结构体 flag_h 参数。

```
typedef enum {
    G2D_BLT_NONE_H = 0x0,   /* single source operation */
    /* rop2 */
    /* Fill the target rectangular area with the color associated with index 0 of the physical color palette.
     * For the default physical color palette, this color is black. */
    G2D_BLT_BLACKNESS,  /* blackness */
    G2D_BLT_NOTMERGEPEN,    /* dst = ~(dst + src) */
    G2D_BLT_MASKNOTPEN, /* dst = ~src & dst */
    G2D_BLT_NOTCOPYPEN, /* dst = ~src */
    G2D_BLT_MASKPENNOT, /* dst = src & ~dst */
    G2D_BLT_NOT,    /* dst = ~dst */
    G2D_BLT_XORPEN, /* dst = src ^ dst */
    G2D_BLT_NOTMASKPEN, /* dst = ~(src & dst) */
    G2D_BLT_MASKPEN,    /* dst = src & dst */
    G2D_BLT_NOTXORPEN,  /* dst = ~(src ^ dst) */
    G2D_BLT_NOP,    /* dst = dst */
    G2D_BLT_MERGENOTPEN,    /* dst = ~src + dst */
    G2D_BLT_COPYPEN,    /* dst = src */
    G2D_BLT_MERGEPENNOT,    /* dst = src + ~dst */
    G2D_BLT_MERGEPEN,   /* dst = src + dst */
    G2D_BLT_WHITENESS,  /* whiteness */

    /* Fill the target rectangular area with the color associated with index 0 of the physical color palette.
     * For the default physical color palette, this color is white. */
    G2D_BLT_EXTRACT = 0x000000ff,

    G2D_ROT_90  = 0x00000100,       //顺时针90度
    G2D_ROT_180 = 0x00000200,       //顺时针180度
    G2D_ROT_270 = 0x00000300,       //顺时针270度
    G2D_ROT_0   = 0x00000400,       //顺时针0度，本质上是memcpy操作
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

```
    G2D_ROT_H = 0x00001000,         //水平镜像
    G2D_ROT_V = 0x00002000,         //垂直镜像
    G2D_ROT_90_H   = 0x00001100,    //顺时针90度+水平镜像
    G2D_ROT_90_V   = 0x00002100,    //顺时针90度+垂直镜像

/*  G2D_SM_TDLR_1  =    0x10000000, */
    G2D_SM_DTLR_1 = 0x10000000,
/*  G2D_SM_TDRL_1  =    0x20000000, */
/*  G2D_SM_DTRL_1  =    0x30000000, */
} g2d_blt_flags_h;
```

g2d_image_enh 结构体代表的是对图层的描述，如果应用层使用 dma_buf_map 方式传递 g2d buffer 地址，则 fd 参数填入对应的文件描述符，若使用物理地址，则 laddr[3] 对应 Y（R）、V、U 三分量的低 32 位起始地址，haddr[3] 对应他们的高 32 位地址（64 位系统）。align 参数用于配置 Y（R）、V、U 三分量的字节对齐，对齐要求可参考 2.4 限制条件章节。旋转功能可与裁剪功能并行处理，可配置 clip 参数实现，具体实现方式可参考操作混合 小节。

```
/* image struct */
typedef struct {
    int      bbuff; /* determine color_filling or image buffer */
    __u32        color; /* color_filling */
    g2d_fmt_enh  format;    /* pixel format */
    __u32        laddr[3];  /* low address for three planes */
    __u32        haddr[3];  /* high address for three planes */
    __u32        width; /* image window width */
    __u32        height;    /* image window height */
    __u32        align[3];  /* image windows alignment, y/u/v planes*/

    g2d_rect     clip_rect; /* roi */
    g2d_size     resize;    /* image size after down_sampling */
    g2d_coor     coor;  /* top-left corner coordinates placed in the dst_image window
                         * after src_image has been processed */

    g2d_color_gmt    gamut; /* color space */
/*  __u32        gamut; */
    int      bpremul;   /* alpha premutiled or not */
    __u8         alpha;
    g2d_alpha_mode_enh mode;    /* alpha mode */
    int      fd;
    __u32 use_phy_addr; /* use phy_addr or not */
    enum color_range color_range;
    g2d_fc_cfg_t fc_cfg;
} g2d_image_enh;
```

5.1.2 使用 demo

技巧

目前支持的方向请参考第三章概述的旋转和镜像小节。

将格式为 argb8888 格式，分辨率为 1920x1080 分辨率图像旋转 90 度

```
int ret;
int g2d_fd;
g2d_blt_h blit;

g2d_fd = open("dev/g2d", O_RDWR);
if (g2d_fd < 0) {
    printf("failed to open g2d device\n");
    return;
}
memset(&blit, 0, sizeof(g2d_blt_h));

blit.flag_h = G2D_ROT_90;

#ifdef USE_PHY_ADDR
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
blit.src_image_h.use_phy_addr = 1;
blit.src_image_h.laddr[0] = src_image_addr[0];
#elif USE_DMA_BUFFERR_HEAP
blit.src_image_h.use_phy_addr = 0;
blit.src_image_h.fd = src_image_fd;
#else
...
#endif
blit.src_image_h.bbuff = 1;
blit.src_image_h.mode = G2D_PIXEL_ALPHA;
blit.src_image_h.alpha = 0xff;
blit.src_image_h.format = G2D_FORMAT_ARGB8888;
blit.src_image_h.width = 1920;
blit.src_image_h.height = 1080;
blit.src_image_h.clip_rect.x = 0;
blit.src_image_h.clip_rect.y = 0;
blit.src_image_h.clip_rect.w = 1920;
blit.src_image_h.clip_rect.h = 1080;

#ifdef USE_PHY_ADDR
blit.dst_image_h.use_phy_addr = 1;
blit.dst_image_h.laddr[0] = dst_image_addr[0];
#elif USE_DMA_BUFFERR_HEAP
blit.dst_image_h.use_phy_addr = 0;
blit.dst_image_h.fd = dst_image_fd;
#else
...
#endif
blit.dst_image_h.bbuff = 1;
blit.dst_image_h.mode = G2D_PIXEL_ALPHA;
blit.dst_image_h.alpha = 0xff;
blit.dst_image_h.format = G2D_FORMAT_ARGB8888;
blit.dst_image_h.width = 1080;
blit.dst_image_h.height = 1920;
blit.dst_image_h.clip_rect.x = 0;
blit.dst_image_h.clip_rect.y = 0;
blit.dst_image_h.clip_rect.w = 1080;
blit.dst_image_h.clip_rect.h = 1920;

ret = ioctl(g2d_fd, G2D_CMD_BITBLT_H ,(unsigned long)(&blit))
```

5.2 图像裁剪和拼接 (Crop/Stiching)

5.2.1 关键参数配置

裁剪功能依赖于对图层取数的参数设置，重点与输入图像 src 的 width, height, clip_rect.x, clip_rect.y, clip_rect.w, clip_rect.h 参数有关，注意可以把 width 和 height 理解为图像处理领域中整个图像区域的 window_size，clip_rect 等价于 ROI 。拼接功能依赖于回写时候的参数设置，重点与输出图像 dst 的 width, height, clip_rect.x, clip_rect.y, clip_rect.w, clip_rect.h 参数有关。其余参数的解释参考本章旋转和镜像 (Rotate/Mirror) 的关键参数配置小节。

```
/* image struct */
typedef struct {
    int      bbuff;
    __u32        color;
    g2d_fmt_enh  format;    /* pixel format */
    __u32        laddr[3];  /* low address for three planes */
    __u32        haddr[3];  /* high address for three planes */
    __u32        width; /* image window width */
    __u32        height;    /* image window height */
    __u32        align[3];  /* image windows alignment, y/u/v planes*/

    g2d_rect     clip_rect; /* roi */
    g2d_size     resize;    /* image size after down_sampling */
    g2d_coor     coor;  /* top-left corner coordinates placed in the dst_image window
                         * after src_image has been processed */
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
    g2d_color_gmt    gamut; /* color space */
/*  __u32        gamut; */
    int      bpremul;   /* alpha premutiled or not */
    __u8         alpha;
    g2d_alpha_mode_enh mode;   /* alpha mode */
    int      fd;
    __u32 use_phy_addr; /* use phy_addr or not */
    enum color_range color_range;
    g2d_fc_cfg_t fc_cfg;
} g2d_image_enh;
```

5.2.2 使用 demo

由于裁剪拼接任务往往常见于将一张大图切割为许多张小图，将多张小图拼接为一张大图，且每帧任务往往参数基本相同，故而如果使用的模块带 rcq 更新，推荐利用 rcq 特性在批处理工作模式下执行此类任务，关于模块是否支持 rcq 请查询设计指标。flag_h 参数使用 G2D_ROT_0 或者 G2D_BLT_NONE_H 都可以。

将格式为 argb8888 格式，分辨率为 1024x1024 分辨率图像 src_image_fd 中 ROI 区域为 (0，0，512，512)、(0，512，512，512)、(512，0，512，512)、(512，512，512，512) 的区域裁剪下来，输出到四个 512*512 分辨率的图像 dst_image_fd[4] 上去。

```
/* 以使用dmabufferheap内存句柄为例 */
#define FRAME_TO_BE_PROCESS 4
int g2d_fd;
unsigned long arg[2];
struct mixer_para info[FRAME_TO_BE_PROCESS];
int task_id
int ret;

g2d_fd = open("dev/g2d", O_RDWR);
if (g2d_fd < 0) {
    printf("failed to open g2d device\n");
    return;
}
memset(&blt, 0, sizeof(g2d_blt_h));

info[0].op_flag = OP_BITBLT;
info[0].flag_h = G2D_BLT_NONE_H;
info[0].src_image_h.use_phy_addr = 0;
info[0].src_image_h.fd = src_image_fd;
info[0].src_image_h.bbuff = 1;
info[0].src_image_h.mode = G2D_PIXEL_ALPHA;
info[0].src_image_h.alpha = 0xff;
info[0].src_image_h.format = G2D_FORMAT_ARGB8888;
info[0].src_image_h.width = 1024;
info[0].src_image_h.height = 1024;
info[0].src_image_h.clip_rect.x = 0;
info[0].src_image_h.clip_rect.y = 0;
info[0].src_image_h.clip_rect.w = 512;
info[0].src_image_h.clip_rect.h = 512;
info[0].dst_image_h.use_phy_addr = 0;
info[0].dst_image_h.fd = dst_image_fd[0];
info[0].dst_image_h.bbuff = 1;
info[0].dst_image_h.mode = G2D_PIXEL_ALPHA;
info[0].dst_image_h.alpha = 0xff;
info[0].dst_image_h.format = G2D_FORMAT_ARGB8888;
info[0].dst_image_h.width = 512;
info[0].dst_image_h.height = 512;
info[0].dst_image_h.clip_rect.x = 0;
info[0].dst_image_h.clip_rect.y = 0;
info[0].dst_image_h.clip_rect.w = 512;
info[0].dst_image_h.clip_rect.h = 512;

info[1].op_flag = OP_BITBLT;
info[1].flag_h = G2D_BLT_NONE_H;
info[1].src_image_h.use_phy_addr = 0;
info[1].src_image_h.fd = src_image_fd;
info[1].src_image_h.bbuff = 1;
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

```
info[1].src_image_h.mode = G2D_PIXEL_ALPHA;
info[1].src_image_h.alpha = 0xff;
info[1].src_image_h.format = G2D_FORMAT_ARGB8888;
info[1].src_image_h.width = 1024;
info[1].src_image_h.height = 1024;
info[1].src_image_h.clip_rect.x = 0;
info[1].src_image_h.clip_rect.y = 512;
info[1].src_image_h.clip_rect.w = 512;
info[1].src_image_h.clip_rect.h = 512;
info[1].dst_image_h.use_phy_addr = 0;
info[1].dst_image_h.fd = dst_image_fd[1];
info[1].dst_image_h.bbuff = 1;
info[1].dst_image_h.mode = G2D_PIXEL_ALPHA;
info[1].dst_image_h.alpha = 0xff;
info[1].dst_image_h.format = G2D_FORMAT_ARGB8888;
info[1].dst_image_h.width = 512;
info[1].dst_image_h.height = 512;
info[1].dst_image_h.clip_rect.x = 0;
info[1].dst_image_h.clip_rect.y = 0;
info[1].dst_image_h.clip_rect.w = 512;
info[1].dst_image_h.clip_rect.h = 512;

info[2].op_flag = OP_BITBLT;
info[2].flag_h = G2D_BLT_NONE_H;
info[2].src_image_h.use_phy_addr = 0;
info[2].src_image_h.fd = src_image_fd;
info[2].src_image_h.bbuff = 1;
info[2].src_image_h.mode = G2D_PIXEL_ALPHA;
info[2].src_image_h.alpha = 0xff;
info[2].src_image_h.format = G2D_FORMAT_ARGB8888;
info[2].src_image_h.width = 1024;
info[2].src_image_h.height = 1024;
info[2].src_image_h.clip_rect.x = 512;
info[2].src_image_h.clip_rect.y = 0;
info[2].src_image_h.clip_rect.w = 512;
info[2].src_image_h.clip_rect.h = 512;
info[2].dst_image_h.use_phy_addr = 0;
info[2].dst_image_h.fd = dst_image_fd[2];
info[2].dst_image_h.bbuff = 1;
info[2].dst_image_h.mode = G2D_PIXEL_ALPHA;
info[2].dst_image_h.alpha = 0xff;
info[2].dst_image_h.format = G2D_FORMAT_ARGB8888;
info[2].dst_image_h.width = 512;
info[2].dst_image_h.height = 512;
info[2].dst_image_h.clip_rect.x = 0;
info[2].dst_image_h.clip_rect.y = 0;
info[2].dst_image_h.clip_rect.w = 512;
info[2].dst_image_h.clip_rect.h = 512;

info[3].op_flag = OP_BITBLT;
info[3].flag_h = G2D_BLT_NONE_H;
info[3].src_image_h.use_phy_addr = 0;
info[3].src_image_h.fd = src_image_fd;
info[3].src_image_h.bbuff = 1;
info[3].src_image_h.mode = G2D_PIXEL_ALPHA;
info[3].src_image_h.alpha = 0xff;
info[3].src_image_h.format = G2D_FORMAT_ARGB8888;
info[3].src_image_h.width = 1024;
info[3].src_image_h.height = 1024;
info[3].src_image_h.clip_rect.x = 512;
info[3].src_image_h.clip_rect.y = 512;
info[3].src_image_h.clip_rect.w = 512;
info[3].src_image_h.clip_rect.h = 512;
info[3].dst_image_h.use_phy_addr = 0;
info[3].dst_image_h.fd = dst_image_fd[3];
info[3].dst_image_h.bbuff = 1;
info[3].dst_image_h.mode = G2D_PIXEL_ALPHA;
info[3].dst_image_h.alpha = 0xff;
info[3].dst_image_h.format = G2D_FORMAT_ARGB8888;
info[3].dst_image_h.width = 512;
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
info[3].dst_image_h.height = 512;
info[3].dst_image_h.clip_rect.x = 0;
info[3].dst_image_h.clip_rect.y = 0;
info[3].dst_image_h.clip_rect.w = 512;
info[3].dst_image_h.clip_rect.h = 512;

arg[0] = (unsigned long) info;
arg[1] =  FRAME_TO_BE_PROCESS;
task_id = ioctl(g2d_fd, G2D_CMD_CREATE_TASK ,(unsigned long)(arg))
if (task_id < 0)
    printf("failed to create g2d task\n")
memset(arg, 0, sizeof(arg));
arg[0] = task_id;
ret = ioctl(g2d_fd, G2D_CMD_TASK_APPLY ,(unsigned long)(arg))
ret = ioctl(g2d_fd, G2D_CMD_TASK_DESTROY ,(unsigned long)(arg))
```

将 4 个格式为 argb8888 格式，分辨率为 512x512 的图像 src_image_fd[4] 拼接为一个 1024x1024 分辨率的图像 dst_image_fd 以使用 dmabufferheap 内存句柄为例。

```
#define FRAME_TO_BE_PROCESS 4
int g2d_fd;
unsigned long arg[2];
struct mixer_para info[FRAME_TO_BE_PROCESS];
int ret;

g2d_fd = open("dev/g2d", O_RDWR);
if (g2d_fd < 0) {
    printf("failed to open g2d device\n");
    return;
}
memset(&blt, 0, sizeof(g2d_blt_h));

info[0].op_flag = OP_BITBLT;
info[0].flag_h = G2D_BLT_NONE_H;
info[0].src_image_h.use_phy_addr = 0;
info[0].src_image_h.fd = src_image_fd[0];
info[0].src_image_h.bbuff = 1;
info[0].src_image_h.mode = G2D_PIXEL_ALPHA;
info[0].src_image_h.alpha = 0xff;
info[0].src_image_h.format = G2D_FORMAT_ARGB8888;
info[0].src_image_h.width = 0;
info[0].src_image_h.height = 0;
info[0].src_image_h.clip_rect.x = 0;
info[0].src_image_h.clip_rect.y = 0;
info[0].src_image_h.clip_rect.w = 512;
info[0].src_image_h.clip_rect.h = 512;
info[0].dst_image_h.use_phy_addr = 0;
info[0].dst_image_h.fd = dst_image_fd;
info[0].dst_image_h.bbuff = 1;
info[0].dst_image_h.mode = G2D_PIXEL_ALPHA;
info[0].dst_image_h.alpha = 0xff;
info[0].dst_image_h.format = G2D_FORMAT_ARGB8888;
info[0].dst_image_h.width = 1024;
info[0].dst_image_h.height = 1024;
info[0].dst_image_h.clip_rect.x = 0;
info[0].dst_image_h.clip_rect.y = 0;
info[0].dst_image_h.clip_rect.w = 512;
info[0].dst_image_h.clip_rect.h = 512;

info[1].op_flag = OP_BITBLT;
info[1].flag_h = G2D_BLT_NONE_H;
info[1].src_image_h.use_phy_addr = 0;
info[1].src_image_h.fd = src_image_fd[1];
info[1].src_image_h.bbuff = 1;
info[1].src_image_h.mode = G2D_PIXEL_ALPHA;
info[1].src_image_h.alpha = 0xff;
info[1].src_image_h.format = G2D_FORMAT_ARGB8888;
info[1].src_image_h.width = 512;
info[1].src_image_h.height = 512;
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

文档问题反馈: aw-document@allwinnertech.com

```
info[1].src_image_h.clip_rect.x = 0;
info[1].src_image_h.clip_rect.y = 0;
info[1].src_image_h.clip_rect.w = 512;
info[1].src_image_h.clip_rect.h = 512;
info[1].dst_image_h.use_phy_addr = 0;
info[1].dst_image_h.fd = dst_image_fd;
info[1].dst_image_h.bbuff = 1;
info[1].dst_image_h.mode = G2D_PIXEL_ALPHA;
info[1].dst_image_h.alpha = 0xff;
info[1].dst_image_h.format = G2D_FORMAT_ARGB8888;
info[1].dst_image_h.width = 1024;
info[1].dst_image_h.height = 1024;
info[1].dst_image_h.clip_rect.x = 0;
info[1].dst_image_h.clip_rect.y = 512;
info[1].dst_image_h.clip_rect.w = 512;
info[1].dst_image_h.clip_rect.h = 512;

info[2].op_flag = OP_BITBLT;
info[2].flag_h = G2D_BLT_NONE_H;
info[2].src_image_h.use_phy_addr = 0;
info[2].src_image_h.fd = src_image_fd[2];
info[2].src_image_h.bbuff = 1;
info[2].src_image_h.mode = G2D_PIXEL_ALPHA;
info[2].src_image_h.alpha = 0xff;
info[2].src_image_h.format = G2D_FORMAT_ARGB8888;
info[2].src_image_h.width = 512;
info[2].src_image_h.height = 512;
info[2].src_image_h.clip_rect.x = 0;
info[2].src_image_h.clip_rect.y = 0;
info[2].src_image_h.clip_rect.w = 512;
info[2].src_image_h.clip_rect.h = 512;
info[2].dst_image_h.use_phy_addr = 0;
info[2].dst_image_h.fd = dst_image_fd;
info[2].dst_image_h.bbuff = 1;
info[2].dst_image_h.mode = G2D_PIXEL_ALPHA;
info[2].dst_image_h.alpha = 0xff;
info[2].dst_image_h.format = G2D_FORMAT_ARGB8888;
info[2].dst_image_h.width = 1024;
info[2].dst_image_h.height = 1024;
info[2].dst_image_h.clip_rect.x = 512;
info[2].dst_image_h.clip_rect.y = 0;
info[2].dst_image_h.clip_rect.w = 512;
info[2].dst_image_h.clip_rect.h = 512;

info[3].op_flag = OP_BITBLT;
info[3].flag_h = G2D_BLT_NONE_H;
info[3].src_image_h.use_phy_addr = 0;
info[3].src_image_h.fd = src_image_fd[3];
info[3].src_image_h.bbuff = 1;
info[3].src_image_h.mode = G2D_PIXEL_ALPHA;
info[3].src_image_h.alpha = 0xff;
info[3].src_image_h.format = G2D_FORMAT_ARGB8888;
info[3].src_image_h.width = 512;
info[3].src_image_h.height = 512;
info[3].src_image_h.clip_rect.x = 0;
info[3].src_image_h.clip_rect.y = 0;
info[3].src_image_h.clip_rect.w = 512;
info[3].src_image_h.clip_rect.h = 512;
info[3].dst_image_h.use_phy_addr = 0;
info[3].dst_image_h.fd = dst_image_fd;
info[3].dst_image_h.bbuff = 1;
info[3].dst_image_h.mode = G2D_PIXEL_ALPHA;
info[3].dst_image_h.alpha = 0xff;
info[3].dst_image_h.format = G2D_FORMAT_ARGB8888;
info[3].dst_image_h.width = 1024;
info[3].dst_image_h.height = 1024;
info[3].dst_image_h.clip_rect.x = 512;
info[3].dst_image_h.clip_rect.y = 512;
info[3].dst_image_h.clip_rect.w = 512;
info[3].dst_image_h.clip_rect.h = 512;
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

```
arg[0] = (unsigned long) info;
arg[1] =  FRAME_TO_BE_PROCESS;
task_id = ioctl(g2d_fd, G2D_CMD_CREATE_TASK ,(unsigned long)(arg))
if (task_id < 0)
    printf("failed to create g2d task\n")
memset(arg, 0, sizeof(arg));
arg[0] = task_id;
ret = ioctl(g2d_fd, G2D_CMD_TASK_APPLY ,(unsigned long)(arg))
ret = ioctl(g2d_fd, G2D_CMD_TASK_DESTROY ,(unsigned long)(arg))
```

5.3 图像缩放或下采样 (Scaler/Down_Sampling)

5.3.1 关键参数配置

此功能主要通过配置 g2d_image_enh 结构体参数实现。

Scaler 主要和参数 clip_rect 有关，在 src 原图和 dst 输出图片的 clip.w、clip.h 参数不相等时生效，scaler 操作可以和裁剪操作混合，如输入图片分辨率为 1920x1080，配置 src.clip.x=100，src.clip.h=200，src.clip.w=800，src.clip.h=400，dst.clip.w=500，dst.clip.h=300，则 g2d 内部首先从 src 原图，从坐标（100，200）的位置，裁剪出 800x400 分辨率的图层，接着对裁剪后的图层，做 800x400 到 500x300 分辨率的 scaler 缩放，最后输出图片。

Down_Sampling 下采样主要和 resize 参数有关，resize 参数只在 blend 图层混合功能生效，在本章最后一小节 blend 操作混合中介绍。

```
/* image struct */
typedef struct {
    int      bbuff;
    __u32        color;
    g2d_fmt_enh  format;    /* pixel format */
    __u32        laddr[3];  /* low address for three planes */
    __u32        haddr[3];  /* high address for three planes */
    __u32        width; /* image window width */
    __u32        height;    /* image window height */
    __u32        align[3];  /* image windows alignment, y/u/v planes*/

    g2d_rect     clip_rect; /* roi */
    g2d_size     resize;    /* image size after down_sampling */
    g2d_coor     coor;  /* top-left corner coordinates placed in the dst_image window
                         * after src_image has been processed */

    g2d_color_gmt    gamut; /* color space */
    int      bpremul;   /* alpha premutiled or not */
    __u8         alpha;
    g2d_alpha_mode_enh mode;    /* alpha mode */
    int      fd;
    __u32 use_phy_addr; /* use phy_addr or not */
    enum color_range color_range;
    g2d_fc_cfg_t fc_cfg;
} g2d_image_enh;
```

5.3.2 使用 demo

将格式为 argb8888 格式，分辨率为 8192x8192 分辨率的图像下采样到 2048x8192，再缩放为 1920x1080。

```
/* 以使用dmabufferheap内存句柄为例 */
int g2d_fd;
g2d_blt_h blit;
int ret;

g2d_fd = open("dev/g2d", O_RDWR);
if (g2d_fd < 0) {
    printf("failed to open g2d device\n");
    return;
}
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

```
memset(&blt, 0, sizeof(g2d_blt_h));

blit.flag_h = G2D_BLT_NONE_H;

blit.src_image_h.use_phy_addr = 0;
blit.src_image_h.fd = src_image_fd;
blit.src_image_h.bbuff = 1;
blit.src_image_h.mode = G2D_PIXEL_ALPHA;
blit.src_image_h.alpha = 0xff;
blit.src_image_h.format = G2D_FORMAT_ARGB8888;
blit.src_image_h.width = 8192;
blit.src_image_h.height = 8192;
/* 如果不需要下采样，不需要设置resize参数 */
blit.src_image_h.resize.w = 2048;
blit.src_image_h.resize.h = 8192;

blit.src_image_h.clip_rect.x = 0;
blit.src_image_h.clip_rect.y = 0;
blit.src_image_h.clip_rect.w = 2048;
blit.src_image_h.clip_rect.h = 8192;

blit.dst_image_h.use_phy_addr = 0;
blit.dst_image_h.fd = dst_image_fd;
blit.dst_image_h.bbuff = 1;
blit.dst_image_h.mode = G2D_PIXEL_ALPHA;
blit.dst_image_h.alpha = 0xff;
blit.dst_image_h.format = G2D_FORMAT_ARGB8888;
blit.dst_image_h.width = 1920;
blit.dst_image_h.height = 1080;
blit.dst_image_h.clip_rect.x = 0;
blit.dst_image_h.clip_rect.y = 0;
blit.dst_image_h.clip_rect.w = 1920;
blit.dst_image_h.clip_rect.h = 1080;

ret = ioctl(g2d_fd, G2D_CMD_BITBLT_H ,(unsigned long)(&blit))
```

5.4 图像格式转换（Format_Conversion）

5.4.1 关键参数配置

格式转换的配置主要和输入输出图像 g2d_image_enh 结构体成员的 format 参数有关，g2d_fmt_enh 是输入输出图片格式类型的枚举，各平台支持格式参考平台支持列表与设计指标小节。

```
/* g2d data format for interface 2.0 */
typedef enum {
    G2D_FORMAT_ARGB8888, /* MSB A_R_G_B LSB */
    G2D_FORMAT_ABGR8888,
    G2D_FORMAT_RGBA8888,
    G2D_FORMAT_BGRA8888,
    G2D_FORMAT_XRGB8888,
    G2D_FORMAT_XBGR8888,
    G2D_FORMAT_RGBX8888,
    G2D_FORMAT_BGRX8888,
    G2D_FORMAT_RGB888,
    G2D_FORMAT_BGR888,
    G2D_FORMAT_RGB565,
    G2D_FORMAT_BGR565,
    G2D_FORMAT_ARGB4444,
    G2D_FORMAT_ABGR4444,
    G2D_FORMAT_RGBA4444,
    G2D_FORMAT_BGRA4444,
    G2D_FORMAT_ARGB1555,
    G2D_FORMAT_ABGR1555,
    G2D_FORMAT_RGBA5551,
    G2D_FORMAT_BGRA5551,
    G2D_FORMAT_ARGB2101010 = 0x14,
    G2D_FORMAT_ABGR2101010,
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

```
    G2D_FORMAT_RGBA1010102,
    G2D_FORMAT_BGRA1010102,
    G2D_FORMAT_ARGB8685 = 0x18,
    G2D_FORMAT_ABGR8565,
    G2D_FORMAT_RGBA5658,
    G2D_FORMAT_BGRA5658,

    /* invailed for UI channel */
    G2D_FORMAT_IYUV422_V0Y1U0Y0 = 0x20,
    G2D_FORMAT_IYUV422_Y1V0Y0U0,
    G2D_FORMAT_IYUV422_U0Y1V0Y0,
    G2D_FORMAT_IYUV422_Y1U0Y0V0,

    G2D_FORMAT_YUV422UVC_V1U1V0U0, /* nv16 when LSB, nv61 when MSB */
    G2D_FORMAT_YUV422UVC_U1V1U0V0, /* nv61 when LSB, nv16 when MSB */
    G2D_FORMAT_YUV422_PLANAR, /* I422 */

    G2D_FORMAT_YUV420UVC_V1U1V0U0 = 0x28, /* nv12 when LSB, nv21 when MSB */
    G2D_FORMAT_YUV420UVC_U1V1U0V0, /* nv21 when LSB, nv12 when MSB */
    G2D_FORMAT_YUV420_PLANAR, /* I420 */

    G2D_FORMAT_YUV411UVC_V1U1V0U0 = 0x2c,
    G2D_FORMAT_YUV411UVC_U1V1U0V0,
    G2D_FORMAT_YUV411_PLANAR,

    G2D_FORMAT_Y8 = 0x30,

    /* YUV 10bit format , only for rotate/flip*/
    G2D_FORMAT_YUV444UVC_V1U1V0U0 = 0x31, /* NV24 */
    G2D_FORMAT_YUV444UVC_U1V1U0V0 = 0x32, /* NV42 */
    G2D_FORMAT_YUV444_PLANAR = 0x33, /* I444 Y...U...V... */
    G2D_FORMAT_YVU10_P010 = 0x34,
    G2D_FORMAT_YUV444_PACKED = 0x35, /* V1U1Y1V0U0Y0 */

    G2D_FORMAT_YVU10_P210 = 0x36,

    G2D_FORMAT_YVU10_444 = 0x38,
    G2D_FORMAT_YUV10_444 = 0x39,

    /* YUV444 format, only for g2d200-mixer */
    G2D_FORMAT_YVU444_PLANAR = 0x40, /* YV24 Y...V...U... */

    G2D_FORMAT_MAX = 0xff,
} g2d_fmt_enh;

/* image struct */
typedef struct {
    int      bbuff;
    __u32        color;
    g2d_fmt_enh  format;    /* pixel format */
    __u32        laddr[3];  /* low address for three planes */
    __u32        haddr[3];  /* high address for three planes */
    __u32        width; /* image window width */
    __u32        height;    /* image window height */
    __u32        align[3];  /* image windows alignment, y/u/v planes*/

    g2d_rect     clip_rect; /* roi */
    g2d_size     resize;    /* image size after down_sampling */
    g2d_coor     coor;  /* top-left corner coordinates placed in the dst_image window
                         * after src_image has been processed */

    g2d_color_gmt    gamut; /* color space */
    int      bpremul;   /* alpha premutiled or not */
    __u8         alpha;
    g2d_alpha_mode_enh mode;    /* alpha mode */
    int      fd;
    __u32 use_phy_addr; /* use phy_addr or not */
    enum color_range color_range;
    g2d_fc_cfg_t fc_cfg;
} g2d_image_enh;
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

5.4.2 使用 demo

将格式为 argb8888 格式，分辨率为 1920x1080 分辨率的图像转换为 nv12 格式。

```
/* 以使用dmabufferheap内存句柄为例 */
int g2d_fd;
g2d_blt_h blt;
int ret;

g2d_fd = open("dev/g2d", O_RDWR);
if (g2d_fd < 0) {
    printf("failed to open g2d device\n");
    return;
}
memset(&blt, 0, sizeof(g2d_blt_h));

blit.flag_h = G2D_BLT_NONE_H;

blit.src_image_h.use_phy_addr = 0;
blit.src_image_h.fd = src_image_fd;
blit.src_image_h.bbuff = 1;
blit.src_image_h.mode = G2D_PIXEL_ALPHA;
blit.src_image_h.alpha = 0xff;
blit.src_image_h.format = G2D_FORMAT_ARGB8888;
blit.src_image_h.width = 1920;
blit.src_image_h.height = 1080;
blit.src_image_h.clip_rect.x = 0;
blit.src_image_h.clip_rect.y = 0;
blit.src_image_h.clip_rect.w = 1920;
blit.src_image_h.clip_rect.h = 1080;

blit.dst_image_h.use_phy_addr = 0;
blit.dst_image_h.fd = dst_image_fd;
blit.dst_image_h.bbuff = 1;
blit.dst_image_h.mode = G2D_PIXEL_ALPHA;
blit.dst_image_h.alpha = 0xff;
blit.dst_image_h.format = G2D_FORMAT_YUV420UVC_U1V1U0V0;
blit.dst_image_h.width = 1920;
blit.dst_image_h.height = 1080;
blit.dst_image_h.clip_rect.x = 0;
blit.dst_image_h.clip_rect.y = 0;
blit.dst_image_h.clip_rect.w = 1920;
blit.dst_image_h.clip_rect.h = 1080;

ret = ioctl(g2d_fd, G2D_CMD_BITBLT_H ,(unsigned long)(&blit))
```

5.5 图层混合 (Alpha Blending)

5.5.1 关键参数配置

g2d_bld_cmd_flag 是 blend 操作类型枚举。当前 g2d 驱动最多允许两个图层进行 blend 混合操作，图层的结构体描述分别是 src_image[0] 和 src_image[1]，blend 图层混合操作可与其他 g2d 操作如下采样、裁剪操作混合处理，demo 可参考本章最后的操作混合小节。

```
typedef enum {
    G2D_BLD_CLEAR = 0x00000001,
    G2D_BLD_COPY = 0x00000002,
    G2D_BLD_SRC = 0x00000003,   /* result = source */
    G2D_BLD_DST = 0x00000003,   /* result = destination */
    G2D_BLD_SRCOVER = 0x00000004,   /* result = (1 - As) * destination + source */
    G2D_BLD_DSTOVER = 0x00000005,   /* result = (1 - Ad) * source + destination*/
    G2D_BLD_SRCIN = 0x00000006, /* result = Ad * source */
    G2D_BLD_DSTIN = 0x00000007, /* result = As * destination */
    G2D_BLD_SRCOUT = 0x00000008,    /* result = (1 - Ad) * source */
    G2D_BLD_DSTOUT = 0x00000009,    /* result = (1 - As) * destination */
    G2D_BLD_SRCATOP = 0x0000000a,   /* result = (1 - As) * destination + Ad * source */
    G2D_BLD_DSTATOP = 0x0000000b,   /* result = As * destination + (1 - Ad) * source */
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
    G2D_BLD_XOR = 0x0000000c,   /* result = (1 - As) * destination + (1 - Ad) * source */
    /* when the pixel value matches destination images,
     * it displays the pixel from source image */
    G2D_CK_SRC = 0x00010000,
    /* when the pixel value matches source images,
     * it displays the pixel from destination image */
    G2D_CK_DST = 0x00020000,
} g2d_bld_cmd_flag;

typedef struct {
    g2d_bld_cmd_flag bld_cmd;
    g2d_image_enh dst_image;
    g2d_image_enh src_image[4];/* now only ch0 and ch3 */
    g2d_ck ck_para;
} g2d_bld;          /* blending enhance */
```

5.5.2 使用 demo

将两张 1280x800 的图片进行混合。

```
/* 注：这里的src_image[0]、src_image[1]和destination 相当于struct g2d_bld_cmd_flag 中的source、destination和result */
int ret;
g2d_bld blend;

g2d_fd = open("dev/g2d", O_RDWR);
if (g2d_fd < 0) {
    printf("failed to open g2d device\n");
    return;
}
memset(&blt, 0, sizeof(g2d_bld));

blend.bld_cmd = G2D_BLD_COPY;   /* 根据具体想要的混合效果选择,详情见头文件g2d_bld_cmd_flag */

blend.src_image[0].use_phy_addr = 0;
blend.src_image[0].fd = src_image_fd;
blend.src_image[0].bbuff = 1;
blend.src_image[0].mode = G2D_PIXEL_ALPHA;
blend.src_image[0].format = G2D_FORMAT_ARGB8888;
blend.src_image[0].alpha = 128;
blend.src_image[0].width = 1280;
blend.src_image[0].height = 800;
blend.src_image[0].clip_rect.x = 0;
blend.src_image[0].clip_rect.y = 0;
blend.src_image[0].clip_rect.w = 1280;
blend.src_image[0].clip_rect.h = 800;

blend.src_image[1].use_phy_addr = 0;
blend.src_image[1].fd = src1_image_fd;
blend.src_image[1].bbuff = 1;
blend.src_image[1].mode = G2D_PIXEL_ALPHA;
blend.src_image[1].format = G2D_FORMAT_ARGB8888;
blend.src_image[1].alpha = 128;
blend.src_image[1].width = 1280;
blend.src_image[1].height = 800;
blend.src_image[1].clip_rect.x = 0;
blend.src_image[1].clip_rect.y = 0;
blend.src_image[1].clip_rect.w = 1280;
blend.src_image[1].clip_rect.h = 800;

blend.dst_image.use_phy_addr = 0;
blend.dst_image.fd = dst_image_fd;
blend.dst_image.bbuff = 1;
blend.dst_image.mode = G2D_PIXEL_ALPHA;
blend.dst_image.format = G2D_FORMAT_ARGB8888;
blend.dst_image.alpha = 128;
blend.dst_image.width = 1280;
blend.dst_image.height = 800;
blend.dst_image.clip_rect.x = 0;
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

```
blend.dst_image.clip_rect.y = 0;
blend.dst_image.clip_rect.w = 1280;
blend.dst_image.clip_rect.h = 800;

ret = ioctl(g2d_fd, G2D_CMD_BLD_H ,(unsigned long)(&blend))
```

5.6 画点/画线/矩形填充 (Point/Line Drawing/Rect_ColorFilling)

5.6.1 关键参数配置

关键参数的解释参考如下代码块中的注释：

```
/* g2d_image_enh的color成员用于传递填充的颜色参数，各个分量：A[31:24] R[23:16] G[15:8] B[7:0] */
typedef struct {
    g2d_image_enh dst_image_h;
} g2d_fillrect_h;

/* bbuff用于配置使用图层使用应用层buffer传入的图像还是硬件纯色填充，纯色填充时bbuff参数设为0，反之为1，如果图层选择硬件纯色填充，则color参
     数生效，用于配置纯色填充的A、R、G、B参数 */
typedef struct {
    int      bbuff; /* determine color_filling or image buffer */
    __u32        color; /* color_filling */
    g2d_fmt_enh  format;    /* pixel format */
    __u32        laddr[3];  /* low address for three planes */
    __u32        haddr[3];  /* high address for three planes */
    __u32        width; /* image window width */
    __u32        height;    /* image window height */
    __u32        align[3];  /* image windows alignment, y/u/v planes*/

    g2d_rect     clip_rect; /* roi */
    g2d_size     resize;    /* image size after down_sampling */
    g2d_coor     coor;  /* top-left corner coordinates placed in the dst_image window
                         * after src_image has been processed */

    g2d_color_gmt    gamut; /* color space */
    int      bpremul;   /* alpha premutiled or not */
    __u8         alpha;
    g2d_alpha_mode_enh mode;    /* alpha mode */
    int      fd;
    __u32 use_phy_addr; /* use phy_addr or not */
    enum color_range color_range;
} g2d_image_enh;
```

5.6.2 使用 demo

通过配置下述 demo 的 width, height,clip_rect 即可实现画点画线。

```
int g2d_fd;
g2d_blt_h blt;
int ret;

g2d_fd = open("dev/g2d", O_RDWR);
if (g2d_fd < 0) {
    printf("failed to open g2d device\n");
    return;
}
memset(&blt, 0, sizeof(g2d_blt_h));

blit.flag_h = G2D_BLT_NONE_H;

blit.src_image_h.use_phy_addr = 0;
blit.src_image_h.fd = src_image_fd;
blit.src_image_h.bbuff = 0;
blit.src_image_h.color = 0xffff0000; /* alpha(bit31-bit24) Y/R(bit23-bit16) U/G(bit15-bit8) V/B(bit7-bit0) */
blit.src_image_h.mode = G2D_PIXEL_ALPHA;
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

```
blit.src_image_h.alpha = 0xff;
blit.src_image_h.format = G2D_FORMAT_ARGB8888;
blit.src_image_h.width = 1920;
blit.src_image_h.height = 1080;
blit.src_image_h.clip_rect.x = 0;
blit.src_image_h.clip_rect.y = 0;
blit.src_image_h.clip_rect.w = 1920;
blit.src_image_h.clip_rect.h = 1080;

blit.dst_image_h.use_phy_addr = 0;
blit.dst_image_h.fd = dst_image_fd;
blit.dst_image_h.bbuff = 0;
blit.dst_image_h.mode = G2D_PIXEL_ALPHA;
blit.dst_image_h.alpha = 0xff;
blit.dst_image_h.format = G2D_FORMAT_ARGB8888;
blit.dst_image_h.width = 1920;
blit.dst_image_h.height = 1080;
blit.dst_image_h.clip_rect.x = 0;
blit.dst_image_h.clip_rect.y = 0;
blit.dst_image_h.clip_rect.w = 1920;
blit.dst_image_h.clip_rect.h = 1080;

ret = ioctl(g2d_fd, G2D_CMD_BITBLT_H ,(unsigned long)(&blit))
```

5.7 色键 (ColorKey)

5.7.1 关键结构

```
typedef struct {
    int match_rule; /* 当match_rule为假时，Color Min=<Color<=Color Max表示满足匹配条件;当match_rule为真时，Color>Color Max or Color
     <Color Min表示满足匹配条件 */
    __u32 max_color;    /* Color Max */
    __u32 min_color;    /* Color Min */
}g2d_ck;
```

5.7.2 使用 demo

将两张 1280x800 的图片进行混合, 同时对 src_image[0] 设置色键。

```
/* 注：这里的src_image[0]、src_image[1]和destination 相当于struct g2d_bld_cmd_flag 中的source、destination和result */
int ret;
g2d_bld blend;

g2d_fd = open("dev/g2d", O_RDWR);
if (g2d_fd < 0) {
    printf("failed to open g2d device\n");
    return;
}
memset(&blt, 0, sizeof(g2d_bld));

blend.bld_cmd = G2D_BLD_COPY | G2D_CK_SRC;
blend.ck_para.match_rule = ture;
blend.ck_para.min_color = 35;
blend.ck_para.max_color = 235;

blend.src_image[0].mode = G2D_PIXEL_ALPHA;
blend.src_image[0].format = G2D_FORMAT_ARGB8888;
blend.src_image[0].alpha = 128;
blend.src_image[0].width = 1280;
blend.src_image[0].height = 800;
blend.src_image[0].clip_rect.x = 0;
blend.src_image[0].clip_rect.y = 0;
blend.src_image[0].clip_rect.w = 1280;
blend.src_image[0].clip_rect.h = 800;
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

```
blend.src_image[1].mode = G2D_PIXEL_ALPHA;
blend.src_image[1].format = G2D_FORMAT_ARGB8888;
blend.src_image[1].alpha = 128;
blend.src_image[1].width = 1280;
blend.src_image[1].height = 800;
blend.src_image[1].clip_rect.x = 0;
blend.src_image[1].clip_rect.y = 0;
blend.src_image[1].clip_rect.w = 1280;
blend.src_image[1].clip_rect.h = 800;

blend.dst_image.mode = G2D_PIXEL_ALPHA;
blend.dst_image.format = G2D_FORMAT_ARGB8888;
blend.dst_image.alpha = 128;
blend.dst_image.width = 1280;
blend.dst_image.height = 800;
blend.dst_image.clip_rect.x = 0;
blend.dst_image.clip_rect.y = 0;
blend.dst_image.clip_rect.w = 1280;
blend.dst_image.clip_rect.h = 800;

ret = ioctl(g2d_fd, G2D_CMD_BLD_H ,(unsigned long)(&blend))
```

5.8 二元光栅操作 (Rop2)

5.8.1 关键参数配置

主要跟 struct g2d_blt_h 的 flag_h 成员中的 G2D_BLT_BLACKNESS~G2D_BLT_WHITENESS 枚举有关，G2D 目前支持的 ROP2 操作见第三章模块功能概述小节。

```
typedef struct {
    g2d_blt_flags_h flag_h;
    g2d_image_enh src_image_h;
    g2d_image_enh dst_image_h;
} g2d_blt_h;

typedef enum {
    G2D_BLT_NONE_H = 0x0,   /* single source operation */
    /* rop2 */
    /* Fill the target rectangular area with the color associated with index 0 of the physical color palette.
     * For the default physical color palette, this color is black. */
    G2D_BLT_BLACKNESS,  /* blackness */
    G2D_BLT_NOTMERGEPEN,    /* dst = ~(dst + src) */
    G2D_BLT_MASKNOTPEN, /* dst = ~src & dst */
    G2D_BLT_NOTCOPYPEN, /* dst = ~src */
    G2D_BLT_MASKPENNOT, /* dst = src & ~dst */
    G2D_BLT_NOT,    /* dst = ~dst */
    G2D_BLT_XORPEN, /* dst = src ^ dst */
    G2D_BLT_NOTMASKPEN, /* dst = ~(src & dst) */
    G2D_BLT_MASKPEN,    /* dst = src & dst */
    G2D_BLT_NOTXORPEN,  /* dst = ~(src ^ dst) */
    G2D_BLT_NOP,    /* dst = dst */
    G2D_BLT_MERGENOTPEN,    /* dst = ~src + dst */
    G2D_BLT_COPYPEN,    /* dst = src */
    G2D_BLT_MERGEPENNOT,    /* dst = src + ~dst */
    G2D_BLT_MERGEPEN,   /* dst = src + dst */
    G2D_BLT_WHITENESS,  /* whiteness */

    /* Fill the target rectangular area with the color associated with index 0 of the physical color palette.
     * For the default physical color palette, this color is white. */
    G2D_BLT_EXTRACT = 0x000000ff,

    G2D_ROT_90  = 0x00000100,
    G2D_ROT_180 = 0x00000200,
    G2D_ROT_270 = 0x00000300,
    G2D_ROT_0   = 0x00000400,
    G2D_ROT_H = 0x00001000,
    G2D_ROT_V = 0x00002000,
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

文档问题反馈: aw-document@allwinnertech.com

```
    G2D_ROT_90_H  = 0x00001100,
    G2D_ROT_90_V  = 0x00002100,

/*  G2D_SM_TDLR_1  =    0x10000000, */
    G2D_SM_DTLR_1 = 0x10000000,
/*  G2D_SM_TDRL_1  =    0x20000000, */
/*  G2D_SM_DTRL_1  =    0x30000000, */
} g2d_blt_flags_h;
```

5.8.2 使用 demo

```
/* 二元光栅操作 */
int ret;
int g2d_fd;
g2d_blt_h blt;

g2d_fd = open("/dev/g2d", O_RDWR);
if (g2d_fd < 0) {
    printf("failed to open g2d device\n");
    return;
}
memset(&blt, 0, sizeof(g2d_blt_h));

blit.flag_h = G2D_BLT_NOTCOPYPEN;  /* 将原来的图像取反 */

blit.src_image_h.use_phy_addr = 0;
blit.src_image_h.fd = src_image_fd;
blit.src_image_h.bbuff = 1;
blit.src_image_h.mode = G2D_PIXEL_ALPHA;
blit.src_image_h.alpha = 0xff;
blit.src_image_h.format = G2D_FORMAT_ARGB8888;
blit.src_image_h.width = 1920;
blit.src_image_h.height = 1080;
blit.src_image_h.clip_rect.x = 0;
blit.src_image_h.clip_rect.y = 0;
blit.src_image_h.clip_rect.w = 1920;
blit.src_image_h.clip_rect.h = 1080;


blit.dst_image_h.use_phy_addr = 0;
blit.dst_image_h.fd = dst_image_fd;
blit.dst_image_h.bbuff = 1;
blit.dst_image_h.mode = G2D_PIXEL_ALPHA;
blit.dst_image_h.alpha = 0xff;
blit.dst_image_h.format = G2D_FORMAT_ARGB8888;
blit.dst_image_h.width = 1920;
blit.dst_image_h.height = 1080;
blit.dst_image_h.clip_rect.x = 0;
blit.dst_image_h.clip_rect.y = 0;
blit.dst_image_h.clip_rect.w = 1920;
blit.dst_image_h.clip_rect.h = 1080;

ret = ioctl(g2d_fd, G2D_CMD_BITBLT_H ,(unsigned long)(&blit))
```

5.9 三元光栅操作 (Rop3)

5.9.1 关键参数配置

```
typedef struct {
    g2d_rop3_cmd_flag back_flag;
    g2d_rop3_cmd_flag fore_flag;

    g2d_image_enh dst_image_h;
    g2d_image_enh src_image_h;
    g2d_image_enh ptn_image_h;
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

```
    g2d_image_enh mask_image_h;

} g2d_maskblt;

/* ROP3 command */
typedef enum {
    G2D_ROP3_BLACKNESS = 0x00,  /* dst = BLACK */
    G2D_ROP3_NOTSRCERASE = 0x11,    /* dst = (NOT src) AND (NOT dst) */
    G2D_ROP3_NOTSRCCOPY = 0x33, /* dst = (NOT src) */
    G2D_ROP3_SRCERASE = 0x44,   /* dst = src AND (NOT dst ) */
    G2D_ROP3_DSTINVERT = 0x55,  /* dst = (NOT dst) */
    G2D_ROP3_PATINVERT = 0x5A,  /* dst = pattern XOR dst */
    G2D_ROP3_SRCINVERT = 0x66,  /* dst = src XOR dst */
    G2D_ROP3_SRCAND = 0x88, /* dst = src AND dst */
    G2D_ROP3_MERGEPAINT = 0xBB, /* dst = (NOT src) OR dst */
    G2D_ROP3_MERGECOPY = 0xC0,  /* dst = (src AND pattern) */
    G2D_ROP3_SRCCOPY = 0xCC,    /* dst = src */
    G2D_ROP3_SRCPAINT = 0xEE,   /* dst = src OR dst */
    G2D_ROP3_PATCOPY = 0xF0,    /* dst = pattern */
    G2D_ROP3_PATPAINT = 0xFB,   /* dst = (~src OR pattern) OR DST */
    G2D_ROP3_WHITENESS = 0xFF,  /* dst = WHITE */
} g2d_rop3_cmd_flag;
```

5.9.2 使用 demo

G2D 目前支持的 ROP3 操作见模块功能概述

```
/* 三元光栅操作：遮罩操作 */
int ret;
g2d_maskblt mask;

g2d_fd = open("dev/g2d", O_RDWR);
if (g2d_fd < 0) {
    printf("failed to open g2d device\n");
    return;
}
memset(&g2d_maskblt, 0, sizeof(g2d_maskblt));

mask.back_flag = G2D_ROP3_NOTSRCCOPY;
mask.fore_flag = G2D_ROP3_SRCINVERT;
mask.src_image_h.clip_rect.x = 0;
mask.src_image_h.clip_rect.y = 0;
mask.src_image_h.clip_rect.w = 1280;
mask.src_image_h.clip_rect.h = 800;
mask.src_image_h.width = 1280;
mask.src_image_h.height = 800;
mask.src_image_h.mode = G2D_PIXEL_ALPHA;
mask.dst_image_h.clip_rect.x = 0;
mask.dst_image_h.clip_rect.y = 0;
mask.dst_image_h.clip_rect.w = 1280;
mask.dst_image_h.clip_rect.h = 800;
mask.dst_image_h.width = 1280;
mask.dst_image_h.height = 800;
mask.dst_image_h.mode = G2D_PIXEL_ALPHA;
mask.mask_image_h.clip_rect.x = 0;
mask.mask_image_h.clip_rect.y = 0;
mask.mask_image_h.clip_rect.w = 1280;
mask.mask_image_h.clip_rect.h = 800;
mask.mask_image_h.width = 1280;
mask.mask_image_h.height = 800;
mask.mask_image_h.mode = G2D_PIXEL_ALPHA;
mask.ptn_image_h.clip_rect.x = 0;
mask.ptn_image_h.clip_rect.y = 0;
mask.ptn_image_h.clip_rect.w = 1280;
mask.ptn_image_h.clip_rect.h = 800;
mask.ptn_image_h.width = 1280;
mask.ptn_image_h.height = 800;
mask.ptn_image_h.mode = G2D_PIXEL_ALPHA;
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

```
mask.src_image_h.alpha = 0xff;
mask.mask_image_h.alpha = 0xff;
mask.ptn_image_h.alpha = 0xff;
mask.dst_image_h.alpha = 0xff;
mask.src_image_h.format = G2D_FORMAT_ARGB8888;
mask.mask_image_h.format = G2D_FORMAT_ARGB8888;
mask.ptn_image_h.format = G2D_FORMAT_ARGB8888;
mask.dst_image_h.format = G2D_FORMAT_ARGB8888;

ret = ioctl(int fd, G2D_CMD_MASK_H ,(unsigned long)(&mask))
```

5.10 批处理工作模式

G2D 的批处理工作模式主要有同步和异步两种。

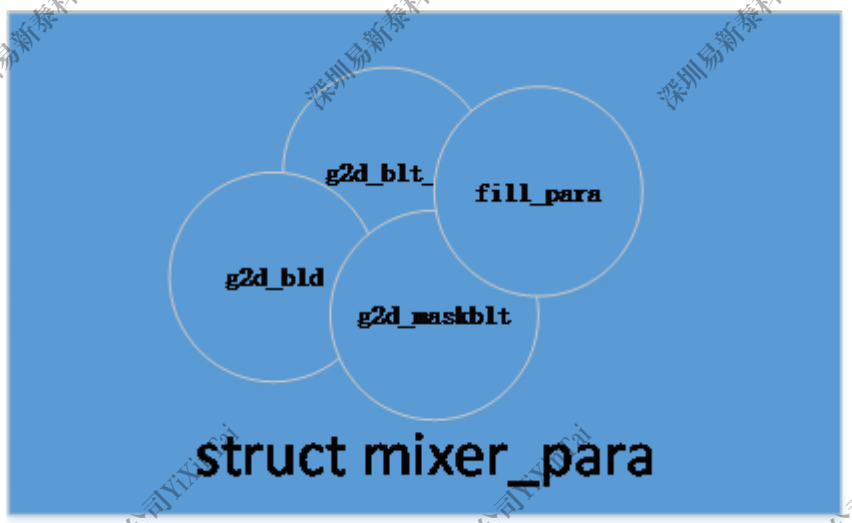
5.10.1 关键参数配置

```
struct mixer_para {
    g2d_operation_flag op_flag;
    g2d_blt_flags_h flag_h;
    g2d_rop3_cmd_flag back_flag;
    g2d_rop3_cmd_flag fore_flag;
    g2d_bld_cmd_flag     bld_cmd;
    g2d_image_enh src_image_h;
    g2d_image_enh dst_image_h;
    g2d_image_enh ptn_image_h;
    g2d_image_enh mask_image_h;
    g2d_ck ck_para;
};

typedef enum {
    OP_FILLRECT = 0x1,
    OP_BITBLT = 0x2,
    OP_BLEND = 0x4,
    OP_MASK = 0x8,
    OP_SPLIT_MEM = 0x10,
} g2d_operation_flag;
```

struct mixer_para 是 RCQ 批处理的核心结构体。可以看到除了第一个成员，其它成员的类型都是旧驱动里面有的。struct mixer_para 是之前驱动接口结构体的一个合集，如图所示：

图 5-1: mixerpara



所以可以用批处理接口完成上面其它接口的功能，只要设置好对应的成员和 g2d_operation_flag 即可。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

文档问题反馈: aw-document@allwinnertech.com

5.10.2 批处理工作同步模式使用 demo

5.10.2.1 G2D_CMD_MIXER_TASK

以下示例为：将格式 argb8888 分辨率为 1024x1024 图像，从 src_image_fd 中 ROI 区域为 (0，0，512，512)、(0，512，512，512)、(512，0，512，512)、(512，512，512，512) 的区域裁剪下来，输出到四个 512*512 分辨率的图像 dst_image_fd[4] 上去。

```
/* 以使用dmabufferheap内存句柄为例 */
#define FRAME_TO_BE_PROCESS 4
int g2d_fd;
unsigned long arg[2];
struct mixer_para info[FRAME_TO_BE_PROCESS];
int ret;

g2d_fd = open("dev/g2d", O_RDWR);
if (g2d_fd < 0) {
    printf("failed to open g2d device\n");
    return;
}
memset(&info, 0, FRAME_TO_BE_PROCESS * sizeof(info));

info[0].op_flag = OP_BITBLT;
info[0].flag_h = G2D_BLT_NONE_H;
info[0].src_image_h.use_phy_addr = 0;
info[0].src_image_h.fd = src_image_fd;
info[0].src_image_h.bbuff = 1;
info[0].src_image_h.mode = G2D_PIXEL_ALPHA;
info[0].src_image_h.alpha = 0xff;
info[0].src_image_h.format = G2D_FORMAT_ARGB8888;
info[0].src_image_h.width = 1024;
info[0].src_image_h.height = 1024;
info[0].src_image_h.clip_rect.x = 0;
info[0].src_image_h.clip_rect.y = 0;
info[0].src_image_h.clip_rect.w = 512;
info[0].src_image_h.clip_rect.h = 512;
info[0].dst_image_h.use_phy_addr = 0;
info[0].dst_image_h.fd = dst_image_fd[0];
info[0].dst_image_h.bbuff = 1;
info[0].dst_image_h.mode = G2D_PIXEL_ALPHA;
info[0].dst_image_h.alpha = 0xff;
info[0].dst_image_h.format = G2D_FORMAT_ARGB8888;
info[0].dst_image_h.width = 512;
info[0].dst_image_h.height = 512;
info[0].dst_image_h.clip_rect.x = 0;
info[0].dst_image_h.clip_rect.y = 0;
info[0].dst_image_h.clip_rect.w = 512;
info[0].dst_image_h.clip_rect.h = 512;

info[1].op_flag = OP_BITBLT;
info[1].flag_h = G2D_BLT_NONE_H;
info[1].src_image_h.use_phy_addr = 0;
info[1].src_image_h.fd = src_image_fd;
info[1].src_image_h.bbuff = 1;
info[1].src_image_h.mode = G2D_PIXEL_ALPHA;
info[1].src_image_h.alpha = 0xff;
info[1].src_image_h.format = G2D_FORMAT_ARGB8888;
info[1].src_image_h.width = 1024;
info[1].src_image_h.height = 1024;
info[1].src_image_h.clip_rect.x = 0;
info[1].src_image_h.clip_rect.y = 512;
info[1].src_image_h.clip_rect.w = 512;
info[1].src_image_h.clip_rect.h = 512;
info[1].dst_image_h.use_phy_addr = 0;
info[1].dst_image_h.fd = dst_image_fd[1];
info[1].dst_image_h.bbuff = 1;
info[1].dst_image_h.mode = G2D_PIXEL_ALPHA;
info[1].dst_image_h.alpha = 0xff;
info[1].dst_image_h.format = G2D_FORMAT_ARGB8888;
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

```
info[1].dst_image_h.width = 512;
info[1].dst_image_h.height = 512;
info[1].dst_image_h.clip_rect.x = 0;
info[1].dst_image_h.clip_rect.y = 0;
info[1].dst_image_h.clip_rect.w = 512;
info[1].dst_image_h.clip_rect.h = 512;

info[2].op_flag = OP_BITBLT;
info[2].flag_h = G2D_BLT_NONE_H;
info[2].src_image_h.use_phy_addr = 0;
info[2].src_image_h.fd = src_image_fd;
info[2].src_image_h.bbuff = 1;
info[2].src_image_h.mode = G2D_PIXEL_ALPHA;
info[2].src_image_h.alpha = 0xff;
info[2].src_image_h.format = G2D_FORMAT_ARGB8888;
info[2].src_image_h.width = 1024;
info[2].src_image_h.height = 1024;
info[2].src_image_h.clip_rect.x = 512;
info[2].src_image_h.clip_rect.y = 0;
info[2].src_image_h.clip_rect.w = 512;
info[2].src_image_h.clip_rect.h = 512;
info[2].dst_image_h.use_phy_addr = 0;
info[2].dst_image_h.fd = dst_image_fd[2];
info[2].dst_image_h.bbuff = 1;
info[2].dst_image_h.mode = G2D_PIXEL_ALPHA;
info[2].dst_image_h.alpha = 0xff;
info[2].dst_image_h.format = G2D_FORMAT_ARGB8888;
info[2].dst_image_h.width = 512;
info[2].dst_image_h.height = 512;
info[2].dst_image_h.clip_rect.x = 0;
info[2].dst_image_h.clip_rect.y = 0;
info[2].dst_image_h.clip_rect.w = 512;
info[2].dst_image_h.clip_rect.h = 512;

info[3].op_flag = OP_BITBLT;
info[3].flag_h = G2D_BLT_NONE_H;
info[3].src_image_h.use_phy_addr = 0;
info[3].src_image_h.fd = src_image_fd;
info[3].src_image_h.bbuff = 1;
info[3].src_image_h.mode = G2D_PIXEL_ALPHA;
info[3].src_image_h.alpha = 0xff;
info[3].src_image_h.format = G2D_FORMAT_ARGB8888;
info[3].src_image_h.width = 1024;
info[3].src_image_h.height = 1024;
info[3].src_image_h.clip_rect.x = 512;
info[3].src_image_h.clip_rect.y = 512;
info[3].src_image_h.clip_rect.w = 512;
info[3].src_image_h.clip_rect.h = 512;
info[3].dst_image_h.use_phy_addr = 0;
info[3].dst_image_h.fd = dst_image_fd[3];
info[3].dst_image_h.bbuff = 1;
info[3].dst_image_h.mode = G2D_PIXEL_ALPHA;
info[3].dst_image_h.alpha = 0xff;
info[3].dst_image_h.format = G2D_FORMAT_ARGB8888;
info[3].dst_image_h.width = 512;
info[3].dst_image_h.height = 512;
info[3].dst_image_h.clip_rect.x = 0;
info[3].dst_image_h.clip_rect.y = 0;
info[3].dst_image_h.clip_rect.w = 512;
info[3].dst_image_h.clip_rect.h = 512;

arg[0] = (unsigned long) info;
arg[1] =  FRAME_TO_BE_PROCESS;
ret = ioctl(g2d_fd, G2D_CMD_MIXER_TASK ,(unsigned long)(arg))
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

文档问题反馈: aw-document@allwinnertech.com

5.10.3 批处理工作异步模式使用 demo

5.10.3.1 G2D_CMD_CREATE_TASK

该 ioctl 命令用于创建新的批处理实例，此过程仅完成软件层面的准备工作，不涉及硬件处理。具体包括：为对应帧数构造 rcq 队列内存，对输入输出图像执行 dma map 与 dma umap 操作；操作完成后，会更新 mixer_para 并反馈至应用层。

task_id 具有唯一性，在批处理实例未被销毁前，该 id 会被持续占用。用户可通过此 id 执行后续操作（如设置、销毁实例，或获取当前 mixer_para）。

如下示例将创建两个批处理实例（分别对应不同帧数及输入输出格式），最终生成两个不同的 task id（task0 和 task1）。关于 mixer_para 的构造方式，可参考 G2D_CMD_MIXER_TASK 的示例。

```
config_mixer_para(&test_info.info);
config_mixer_para(&test_info2.info);
arg[0] = (unsigned long)test_info.info;
arg[1] = FRAME_TO_BE_PROCESS;
task0 = ioctl(g2d_fd, G2D_CMD_CREATE_TASK, (arg));
if (task0 < 1) {
    printf("[%d][%s][%s]G2D_CMD_CREATE_TASK failure!\n", __LINE__,
           __FILE__, __FUNCTION__);
}

arg[0] = (unsigned long)test_info2.info;
arg[1] = FRAME_TO_BE_PROCESS2;
task1 = ioctl(g2d_fd, G2D_CMD_CREATE_TASK, (arg));
if (task1 < 1) {
    printf("[%d][%s][%s]G2D_CMD_CREATE_TASK failure!\n", __LINE__,
           __FILE__, __FUNCTION__);
}
```

5.10.3.2 G2D_CMD_TASK_APPLY

- PROTOTYPE

```
int ioctl(int fd, int cmd, void *arg)
```

- ARGUMENTS

```
cmd             G2D_CMD_TASK_APPLY

arg[0]          task id(由G2D_CMD_CREATE_TASK命令获得 )
```

- RETURN

```
成功：0，失败：失败号
```

该 ioctl 命令的作用是执行批处理的硬件操作。

值得注意 arg[1] 中的 mixer_para，必须是 G2D_CMD_CREATE_TASK 之后返回的 mixer_para 或者是通过另外一个 ioctl 命令 G2D_CMD_TASK_GET_PARA 才行，这里不需要制定帧数的原因是前面的 G2D_CMD_CREATE_TASK 已经指定好帧数，而 G2D_CMD_TASK_APPLY 是基于 task id 来执行的。

```
arg[0] = task0;
if(ioctl(g2d_fd, G2D_CMD_TASK_APPLY, (arg)) < 0) {
    printf("[%d][%s][%s]G2D_CMD_TASK_APPLY failure!\n", __LINE__,
           __FILE__, __FUNCTION__);
    goto FREE_SRC;
}

arg[0] = task1;
if(ioctl(g2d_fd, G2D_CMD_TASK_APPLY, (arg)) < 0) {
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
    printf("[%d][%s][%s]G2D_CMD_TASK_APPLY failure!\n", __LINE__,
           __FILE__, __FUNCTION__);
    goto FREE_SRC;
}
```

5.10.3.3 G2D_CMD_TASK_DESTROY

- PROTOTYPE

```
int ioctl(int fd, int cmd, void *arg)
```

- ARGUMENTS

```
cmd             G2D_CMD_TASK_DESTROY

arg[0]          task id
```

- RETURN

```
成功：0，失败：失败号
```

该 ioctl 命令的作用是销毁指定 task id 的批处理实例。

```
arg[0] = task0;
ret = ioctl(g2d_fd, G2D_CMD_TASK_DESTROY ,(unsigned long)(arg))
```

5.10.3.4 G2D_CMD_TASK_GET_PARA

- PROTOTYPE

```
int ioctl(int fd, int cmd, void *arg)
```

- ARGUMENTS

```
cmd             G2D_CMD_TASK_DESTROY

arg[0]          task id

arg[1]          指向mixer_para指针，多帧的话就是数组指针。
```

- RETURN

```
成功：0，失败：失败号
```

该 ioctl 命令的作用是获取指定 task id 的 mixer para。

用户必须自行保证传入的指针所指向的内存足够存放这么多帧的参数。

5.11 操作混合

rotate 功能仅可以和 crop 操作混合，不支持和其他操作一起做，可参考如下 demo:

demo 中 src 图像的 clip（x，y，w，h）参数配置为（0，0，400，240），从原图的（0，0）坐标裁剪宽 400pixel、高 240pixel 的图层，再进行 90 度旋转。

```
int ret;
int g2d_fd;
g2d_blt_h blit;
```

深圳易新泰科技有限公司YiXinTai

```
g2d_fd = open("dev/g2d", O_RDWR);
if (g2d_fd < 0) {
    printf("failed to open g2d device\n");
    return;
}
memset(&blit, 0, sizeof(g2d_blt_h));

blit.flag_h = G2D_ROT_90;

#ifdef USE_PHY_ADDR
blit.src_image_h.use_phy_addr = 1;
blit.src_image_h.laddr[0] = src_image_addr[0];
#elif USE_DMA_BUFFERR_HEAP
blit.src_image_h.use_phy_addr = 0;
blit.src_image_h.fd = src_image_fd;
#else
...
#endif
blit.src_image_h.bbuff = 1;
blit.src_image_h.mode = G2D_PIXEL_ALPHA;
blit.src_image_h.alpha = 0xff;
blit.src_image_h.format = G2D_FORMAT_ARGB8888;
blit.src_image_h.width = 800;
blit.src_image_h.height = 480;
blit.src_image_h.clip_rect.x = 0;
blit.src_image_h.clip_rect.y = 0;
blit.src_image_h.clip_rect.w = 400;
blit.src_image_h.clip_rect.h = 240;

#ifdef USE_PHY_ADDR
blit.dst_image_h.use_phy_addr = 1;
blit.dst_image_h.laddr[0] = dst_image_addr[0];
#elif USE_DMA_BUFFERR_HEAP
blit.dst_image_h.use_phy_addr = 0;
blit.dst_image_h.fd = dst_image_fd;
#else
...
#endif
blit.dst_image_h.bbuff = 1;
blit.dst_image_h.mode = G2D_PIXEL_ALPHA;
blit.dst_image_h.alpha = 0xff;
blit.dst_image_h.format = G2D_FORMAT_ARGB8888;
blit.dst_image_h.width = 240;
blit.dst_image_h.height = 400;
blit.dst_image_h.clip_rect.x = 0;
blit.dst_image_h.clip_rect.y = 0;
blit.dst_image_h.clip_rect.w = 240;
blit.dst_image_h.clip_rect.h = 400;

ret = ioctl(g2d_fd, G2D_CMD_BITBLT_H ,(unsigned long)(&blit))
```

图像裁剪拼接、缩放或下采样、图像格式转换、图层混合等操作可以混合起来一起做，但注意 blend 混合操作中，缩放或下采样、图像格式转换仅支持对 v_chn（src0）的输入源进行处理。比如可以对 src0 进行裁剪和下采样后，再和经过裁剪操作的 src1 进行图层混合，最后输出一个指定格式。

下面示例将两张 800x480 的图片进行混合，图片 1（src0）下采样到 600x300，图片 2（src0）从坐标（200，200）裁剪出 w=400，h=200 的图像，两图做 blend，最后输出做格式转换到 I420 格式，参考如下 demo：

```
/* 注：这里的src_image[0]、src_image[1]和destination 相当于struct g2d_bld_cmd_flag 中的source、destination和result */
...
blend.bld_cmd = G2D_BLD_SRCOVER;    /* 根据具体想要的混合效果选择,详情见头文件g2d_bld_cmd_flag */

blend.src_image[0].use_phy_addr = 0;
blend.src_image[0].fd = src_image_fd;
blend.src_image[0].bbuff = 1;
blend.src_image[0].mode =   G2D_GLOBAL_ALPHA;;
blend.src_image[0].format = G2D_FORMAT_ARGB8888;
blend.src_image[0].alpha = 128;
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

```
blend.src_image[0].width = 800;
blend.src_image[0].height = 480;
blend.src_image[0].clip_rect.x = 0;
blend.src_image[0].clip_rect.y = 0;
blend.src_image[0].clip_rect.w = 800;
blend.src_image[0].clip_rect.h = 480;
blend.src_image[0].resize.w = 600;
blend.src_image[0].resize.h = 300;

blend.src_image[1].use_phy_addr = 0;
blend.src_image[1].fd = src1_image_fd;
blend.src_image[1].bbuff = 1;
blend.src_image[1].mode = G2D_GLOBAL_ALPHA;
blend.src_image[1].format = G2D_FORMAT_ARGB8888;
blend.src_image[1].alpha = 128;
blend.src_image[1].width = 800;
blend.src_image[1].height = 480;
blend.src_image[1].clip_rect.x = 200;
blend.src_image[1].clip_rect.y = 200;
blend.src_image[1].clip_rect.w = 400;
blend.src_image[1].clip_rect.h = 200;

blend.dst_image.use_phy_addr = 0;
blend.dst_image.fd = dst_image_fd;
blend.dst_image.bbuff = 1;
blend.dst_image.mode = G2D_GLOBAL_ALPHA;
blend.dst_image.format = G2D_FORMAT_YUV420_PLANAR;
blend.dst_image.alpha = 128;
blend.dst_image.width = 600;
blend.dst_image.height = 300;
blend.dst_image.clip_rect.x = 0;
blend.dst_image.clip_rect.y = 0;
blend.dst_image.clip_rect.w = 600;
blend.dst_image.clip_rect.h = 300;

ret = ioctl(g2d_fd, G2D_CMD_BLD_H ,(unsigned long)(&blend))
```

深圳易新泰科技有限公司YiXinTai

版权所有 © 珠海全志科技股份有限公司。保留一切权利

第 6 章 FAQ

6.1 G2D 理论性能计算公式

G2D 硬件对于各种格式图像的处理性能如下：

```
Rotate:
    for ARGB : 2pixel/cycle
    for RGB565 : 4pixel/cycle
    for YUV420 : 4pixel/cycle
Mixer:
    1pixel/cycle
```

1. G2D 硬件做 rotate 每个时钟周期能够处理的像素数量与图像格式和 size 相关，性能计算较为简单。
2. mixer 性能与图像格式无关，mixer 操作包含了 scaler、crop、fillrect、format convert、blend 等操作，总的来说，遵循输入输出中最大的分辨率计算原则，如将 1920x1080 分辨率图像 scaler 缩小到 800x480 分辨率，G2D 性能的分辨率影响因素就以 1920x1080 为准。若将 1920x1080 分辨率图像 scaler 缩小到 200x100 分辨率，会先对输入做抽取到 1920x800 分辨率，再 scaler 操作到 200x100 分辨率，那么性能的分辨率影响因素以 1920x800 为准。blend 操作会比其他 mixer 操作耗时更长，**总之 mixer 的性能以实测为准。**

举例：

G2D 在执行拷贝时，可以通过以下公式进行计算理论耗时（该功能仅支持数据的拷贝评估），G2D 每个时钟周期能够处理的像素数量和 G2D 频率可以查询设计指标获得：

```
单次拷贝图像耗时 = 图像宽 × 图像高 / G2D每秒能处理的像素数量
= 图像宽 × 图像高 / （G2D每个时钟周期能够处理的像素数量 × G2D频率）
```

一幅 1920 × 1080 大小的 YUV420 图像用 G2D（频率设定为 300M）做旋转的理论耗时是：

$$1920 * 1080 / 4 * 300000000 = 0.001728s$$

而实际在运行操作系统的设备上的耗时与 cpu 占用率、带宽等信息相关。

6.2 G2D 调试节点

G2D 驱动调试节点分两类：

- 功能调试，可用于在 G2D 运行过程中出现异常时使用，主要是查看应用层传递的参数和寄存器配置是否正常。
- 性能调试，主要是对处理耗时进行记录，测试帧数。

通过配置 loglevel 节点打印不同的驱动信息：

loglevel	打印信息
1	驱动事件信息
2	打印应用层 ioctl 调用传递参数
4	此次 g2d 操作的寄存器数据

配置 loglevel 节点命令如下：

```
g2d_test_loglevel -loglevel 4
```

此外提供以下方式打印 g2d 200 驱动操作流程耗时信息：

打开耗时调试节点：

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

```
g2d_test_runtime -runtime 1
```

执行 g2d 操作，会在调用操作结束后打印这次 g2d 操作时间信息：

```
time_info:
g2d use            9261 us
g2d hw_proc use    9172 us
```

- g2d use ： 这一次 ioctl 调用的总耗时。
- hw_proc use：从执行 rcq update 到 rcq finish 中断响应的耗时。包含了 g2d 从内存取数，执行数据处理，并回写到内存的时间。

深圳易新泰科技有限公司YiXinTai

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

著作权声明

版权所有 ©2026 珠海全志科技股份有限公司。保留一切权利。

本文档及内容受著作权法保护，其著作权由珠海全志科技股份有限公司（“全志”）拥有并保留一切权利。

本文档是全志的原创作品和版权财产，未经全志书面许可，任何单位和个人不得擅自摘抄、复制、修改、发表或传播本文档内容的部分或全部，且不得以任何形式传播。

商标声明

、、全志科技、（不完全列举）均为珠海全志科技股份有限公司的商标或者注册商标。在本文档描述的产品中出现的其它商标，产品名称，和服务名称，均由其各自所有人拥有。

免责声明

您购买的产品、服务或特性应受您与珠海全志科技股份有限公司（“全志”）之间签署的商业合同和条款的约束。本文档中描述的全部或部分产品、服务或特性可能不在您所购买或使用的范围内。使用前请认真阅读合同条款和相关说明，并严格遵循本文档的使用说明。您将自行承担任何不当使用行为（包括但不限于如超压，超频，超温使用）造成的不利后果，全志概不负责。

本文档作为使用指导仅供参考。由于产品版本升级或其他原因，本文档内容有可能修改，如有变更，恕不另行通知。全志尽全力在本文档中提供准确的信息，但并不确保内容完全没有错误，因使用本文档而发生损害（包括但不限于间接的、偶然的、特殊的损失）或发生侵犯第三方权利事件，全志概不负责。本文档中的所有陈述、信息和建议并不构成任何明示或暗示的保证或承诺。

本文档未以明示或暗示或其他方式授予全志的任何专利或知识产权。在您实施方案或使用产品的过程中，可能需要获得第三方的权利许可。请您自行向第三方权利人获取相关的许可。全志不承担也不代为支付任何关于获取第三方许可的许可费或版税（专利税）。全志不对您所使用的第三方许可技术做出任何保证、赔偿或承担其他义务。